uc3m | Universidad Carlos III de Madrid

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Computational algorithms for pricing financial derivatives under local volatility models

Jose Enrique Zafra Mena

Jorge Morón Vidal
Francisco Manuel Bernal Martínez

Madrid, 2026

SUMMARY

Keywords:

DEDICATION

CONTENTS

1. INTRODUCTION . . . 1
1.1. Main Objectives . . . 1
1.2. Key Contributions . . . 2
1.3. Scope and Structure . . . 2
2. MATHEMATICAL AND FINANCIAL BACKGROUND . . . 3
2.1. State Of The Art . . . 3
2.2. Mathematical Framework For Option Pricing . . . 4
2.3. Stochastic Calculus Background . . . 5
2.4. The Heston Stochastic Volatility Model . . . 6
2.5. Pricing Via Characteristic Functions . . . 8
2.6. The Fourier-COS method . . . 9
2.6.1. The COS Method Applied To The Heston Model . . . 10
2.6.2. Semi-Analytical Greeks from the Heston Characteristic Function . . . 11
2.7. Implied Volatility And Numerical Inversion . . . 12
3. NEURAL NETWORK PRICER FOR HESTON . . . 14
3.1. Problem Formulation . . . 14
3.2. Synthetic Data Formulation . . . 15
3.2.1. Parameter Ranges And Constraints . . . 15
3.2.2. Dataset Splitting And Preprocessing . . . 16
3.2.3. Numerical Label Generation (COS + Brent) . . . 17
3.3. Neural Network Architecture . . . 17
3.3.1. Model Class . . . 18
3.3.2. Architecture Details . . . 18
3.3.3. Output Interpretation . . . 19
3.4. Training Procedure . . . 19
3.4.1. Loss Function . . . 20
3.4.2. Optimizer And Hyperparameters . . . 20
3.4.3. Regularization And Callbacks . . . 21

4. ANN-ACCELERATED CALIBRATION OF THE HESTON MODEL 23
4.1. Role of Calibration in the Pipeline 23
4.2. Inverse Calibration Problem. 24
4.3. Input Representation: Implied-Volatility Surfaces 25
4.4. Architecture of the Calibration Stage. 26
4.5. Differential Evolution Optimization 27
4.6. Synthetic Calibration Experiment 28
4.7. Regularization and Generalization 29
4.8. Smoothness, Hessian Diagnostics, and Identifiability. 30
4.9. Connection with the Downstream PINN Stage 31
5. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL. 33
5.1. Problem formulation 33
5.1.1. PINN Theoretical Framework. 34
5.2. Heston Pricing PDE 35
5.3. Neural Network Architecture 37
5.3.1. Model Class 37
5.3.2. Architecture Details 37
5.3.3. Model Inputs and Outputs 38
5.4. Training Procedure 39
5.4.1. Loss Function. 39
5.4.2. Optimizer and Hyperparameters 40
5.4.3. Sampling strategy 41
5.5. Extension: Sobolev-Enhanced Training for Greek Accuracy 42
5.6. Greeks Computation 43
5.6.1. Definition of Greeks from the PINN Output 43
5.6.2. Automatic Differentiation of the PINN. 44
6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING 46
6.1. Experimental Setup And Evaluation Protocol 46
6.2. ANN Pricer Performance 47
6.2.1. Global Error Metrics 47
6.2.2. Regional Error Profile 49

6.3. Calibration Network Performance 50
6.3.1. Benchmark Snapshot 50
6.3.2. Fit Quality And Convergence 50
6.3.3. Parameter Recovery Profile 51
6.3.4. Local Smoothness And Curvature Study 52
6.4. PINN Pricing Performance 54
6.4.1. Reproducible Benchmark Setup 54
6.4.2. Global Pricing Metrics 55
6.4.3. Spatial And Distributional Diagnostics 55
6.5. PINN Greeks Performance 59
6.5.1. Ablation Design And Reproducible Protocol 59
6.5.2. Ablation Metrics By Greek 60
6.5.3. Visual Diagnostics 60
6.6. Cross-Model Computational Comparison 65
7. CONCLUSION AND FUTURE WORK 67
7.1. Market Data 67
7.2. Exotic Derivatives 67
7.3. Greeks And Hedging 67
7.4. Model Extension 67
7.5. Machine Learning Improvements 67
BIBLIOGRAPHY 68
A. ANALYSIS OF THE COS METHOD ERROR . . . .
B. CALIBRATION OF THE HESTON MODEL . . . .
C. GLOBAL NOTATION GLOSSARY . . . .

LIST OF FIGURES

1.1 High-level pipeline of the thesis (placeholder). 1

4.1 Conceptual graph of the ANN-accelerated calibration workflow. This figure should later be replaced by a clean pipeline diagram. 24

4.2 Placeholder for the implied-volatility surface used as calibration input. . . 26

4.3 Placeholder for the batched evaluation architecture used by the CaNN stage. 27

4.4 Placeholder for the Differential Evolution convergence trace. 28

4.5 Placeholder for the synthetic calibration quote universe. 29

4.6 Placeholder for the signed residual heatmap $\hat{\sigma}_{\mathrm{IV}} - \sigma_{\mathrm{IV}}^{\mathrm{mkt}}$. 29

4.7 Placeholder for the regularization mini-study in the CaNN calibration. . . 30

4.8 Placeholder for local first- and second-order diagnostics at the calibrated solution. 31

4.9 Placeholder for objective-surface slices around the calibrated optimum. . 31

4.10 Placeholder for the interface between calibration and the PINN pricing/-Greeks stage. 32

6.1 Sensitivity grid for the ANN pricer. Top row: $(\gamma, \bar{v})$, middle row: (ρ, γ), bottom row: $(\kappa, \bar{v})$. Left panels report IV-value maps with white isoclines; right panels report $\log|\mathrm{NN} - \mathrm{Heston}|$. Dashed black lines indicate interpolation limits. 48

6.2 Regional ANN MSE by moneyness bins (left) and maturity bins (right). Curves correspond to train, validation, and test. 49

6.3 Signed residual heatmap over (m, τ) for the CaNN reference calibration (no regularization). Residual defined as $\mathrm{IV}_{\mathrm{pred}} - \mathrm{IV}_{\mathrm{market}}$. Rendering is visually smoothed for readability; values come from the original quote grid. 51

6.4 Jacobian components at $\theta^\star$ for the CaNN reference calibration (no regularization). 53

6.5 Hessian matrix at $\theta^\star$ for the CaNN reference calibration (no regularization). 54

6.6 PINN price predictions versus COS reference values on the benchmark set. 56

6.7 Absolute pricing-error map over the (m, τ) benchmark grid. 57

6.8 Relative absolute pricing-error map over the (m, τ) benchmark grid. . . . 58

6.9 Empirical CDF of absolute pricing errors for the baseline PINN. 58

6.10 Relative absolute-error maps for baseline PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO). 61

6.11 Error histograms for baseline PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO). 62

6.12 Relative absolute-error maps for Sobolev-augmented PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO). 63

6.13 Error histograms for Sobolev-augmented PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO). 64

6.14 Training loss trajectory with phase switch at epoch 4000 (without Sobolev before 4000, with Sobolev from 4000 onward). 65

LIST OF TABLES

2.1 Methodological blocks and role in this thesis 4

2.2 Interpretation of Heston parameters 7

3.1 Input ranges used for synthetic data generation 16

3.2 Neural network architecture used for the ANN pricer 19

3.3 Reference and runner-up runs used in optimizer selection 21

3.4 Training hyperparameters of the selected run (`Liu_like_v01`) 21

4.1 Calibration parameter bounds used in the CaNN stage. 25

4.2 Quote-table representation used in calibration. 26

4.3 Differential Evolution controls used in the calibration pipeline. 28

5.1 PINN architecture used in this chapter 38

6.1 Global ANN implied-volatility errors (baseline configuration). 47

6.2 Representative regional ANN errors on the test split. 49

6.3 Quote universe used in the CaNN benchmark. 50

6.4 Absolute-residual distribution across market quotes (selected run). 51

6.5 Parameter error profile for the reference CaNN calibration (no regularization). 52

6.6 Selected CaNN run versus Liu et al. (absolute parameter errors). 52

6.7 Local-curvature diagnostics at the calibrated solution $\theta^\star$ (selected run). . . 53

6.8 Global PINN pricing metrics versus COS reference. 55

6.9 Reproducible protocol for the Greek ablation benchmark. 59

6.10 Greek metrics for baseline and Sobolev PINN (n = 25,921 per Greek). . . 60

C.1 Global notation glossary: core pricing and model notation . . . .

C.2 Global notation glossary: transform, COS, and IV notation . . . .

1. INTRODUCTION

1.1. Main Objectives

This thesis focuses on the Heston stochastic-volatility framework and aims to evaluate computational methods under a consistent speed-accuracy trade-off perspective. The first objective is to build a reproducible end-to-end pipeline covering data preparation, reference numerical pricing, calibration, surrogate/PINN training, and benchmark execution. This objective is methodological: the full workflow must be easy to rerun and auditable at each stage. add final references to scripts and modules once the pipeline layout is done. if needed, consolidate these references in an appendix table

The second objective is comparative. The thesis develops and evaluates surrogate approaches, including supervised neural surrogates and PINN-based formulations, against numerical references under common experimental conditions. The purpose is to determine where learned approximators provide practical acceleration without losing control of model-consistent behavior.

The third objective is empirical evaluation through quantitative metrics. Performance is assessed using pricing error, Greek error, computational cost, and numerical stability across the parameter regions and contract settings covered by the experiments. The goal is to characterize not only average accuracy, but also robustness and failure modes.

The fourth objective is to establish realistic operational boundaries. The thesis explicitly separates components that are sufficiently mature for applied use from those that remain exploratory, especially in relation to portability, maintenance cost, and reliability under distribution shifts.

Placeholder: High-level pipeline diagram

Market data and synthetic data preparation → Heston reference solvers (COS/PDE + IV inversion) → calibration and learned surrogates (ANN/PINN) → pricing and Greeks benchmarks.

Fig. 1.1. High-level pipeline of the thesis (placeholder).

Latency is treated as an informative benchmark dimension rather than as a production service-level target. Reported timings are interpreted in the context of the current research implementation, without hardware-specific optimization as a primary objective.

1.2. Key Contributions

This thesis makes four main contributions. First, it delivers a reproducible computational framework for Heston-based market calibration, reference pricing, surrogate training, and benchmarking under a unified experimental protocol. This framework is designed to make method comparisons auditable and repeatable.

Second, it provides an empirical comparison between numerical references and neural surrogates, including supervised ANN models and PINN-based formulations, under shared evaluation conditions. The comparison is used to characterize practical speed-accuracy trade-offs and to identify parameter regions where each approach remains reliable.

Third, it develops a PINN-based workflow for fast Greek computation in the vanilla European Heston setting, using automatic differentiation on a differentiable pricing surrogate and validating the resulting sensitivities against reference numerical estimates.

Fourth, it documents the operational limits of the proposed methodology through an explicit robustness and validity analysis, including data-domain coverage, discretization effects, extrapolation risk, and dependence on benchmark design. This yields a clear boundary between components that are mature enough for applied use and components that remain exploratory.

1.3. Scope and Structure

The scope of this thesis is explicitly bounded to Heston-based pricing, calibration, and Greek estimation workflows under the data and contract settings defined in the experimental design. The focus is on methodological evaluation and reproducible benchmarking, not on full production deployment or broad asset-class coverage beyond the tested domain.

Validation is presented at a high level through the datasets used, the train/validation/test logic adopted in each learning stage, and the benchmark criteria applied to numerical and neural methods. The goal is to keep the comparison protocol transparent and consistent across chapters. finalize the benchmark list and the exact data period after the last data-quality review

The remainder of the thesis is organized as follows. Chapter 2 introduces the mathematical and financial background. Chapter 3 presents the supervised neural surrogate pricer under Heston. Chapter 4 covers the calibration architecture and workflow. Chapter 5 develops the PINN formulation for pricing and Greeks. Chapter 6 reports performance and benchmark results. Finally, Chapter 7 summarizes conclusions and future work.

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. State Of The Art

The literature relevant to this thesis can be grouped into three blocks. First, classical model-based pricing combines closed or semi-closed formulas (e.g., Black–Scholes and Heston) with deterministic numerical solvers under explicit stochastic assumptions [1], [2]. These methods are financially interpretable and robust, but repeated calls in calibration or surface-generation loops can be computationally expensive.

Second, spectral/Fourier pricing methods exploit characteristic functions to obtain fast and accurate valuation formulas, with FFT- and COS-based implementations as standard references [3], [4]. Their main strength is the accuracy-speed tradeoff in repeated pricing, while practical performance still depends on stable parameterization choices (truncation interval, series size, and numerical inversion settings).

Third, neural surrogates learn the pricing map directly from synthetic or market-consistent labels. Feed-forward ANN surrogates have shown substantial acceleration in calibration workflows [5]. More recently, PINN-type approaches have been proposed to inject structural constraints and improve data efficiency [6], [7]. However, the PINN literature also reports optimization pathologies, sensitivity to loss balancing, and robustness issues outside the training distribution [8], [9], [10].

Within this landscape, this thesis reuses the Heston-COS framework as high-accuracy numerical reference and contributes an empirical comparison of data-driven alternatives (ANN and PINN-family variants) under a common data-generation, evaluation, and benchmarking protocol.

TABLE 2.1. METHODOLOGICAL BLOCKS AND ROLE IN THIS THESIS

Block	Representative methods	Main strengths	Main limitations
Classical model-based pricing	Black–Scholes, Heston semi-closed pricing [1], [2]	Financial interpretability; strong theoretical grounding	High computational cost in repeated evaluations
Spectral pricing	FFT/Carr–Madan, COS/Fang–Oosterlee [3], [4]	High accuracy with efficient repeated pricing	Requires careful numerical setup ($[a, b]$, N, inversion stability)
Data-driven surrogates	ANN pricers/calibrators [5]	Very fast inference once trained	Domain-shift and extrapolation risk
Physics-informed surrogates	PINN/gPINN variants [6], [7]	Injects structural constraints into learning	Training instability and sensitive optimization [8], [9], [10]

Source: Author's synthesis from the cited literature

2.2. Mathematical Framework For Option Pricing

This section defines the common mathematical setup used in the rest of the thesis for pricing European-style derivatives.

Time is continuous on $[0, T]$, with current time t and maturity T. A derivative is a financial contract whose value depends on an underlying asset price process S_t; in particular, S_T denotes the underlying price at maturity. A European option gives its holder a right, but not an obligation, exercisable only at maturity T: a call option pays when S_T finishes above the strike K, while a put option pays when S_T finishes below K. For example, the terminal payoff is $(S_T - K)^+$ for a European call and $(K - S_T)^+$ for a European put, where $(x)^+ = \max(x, 0)$. Rather than introducing the full probabilistic machinery, we keep a pricing-focused notation that is sufficient for the methods used later in the thesis.

A global notation glossary is provided in Appendix C and will be extended as new symbols are introduced.

As a practical starting point for this thesis, we use the standard risk-neutral valuation formula for European claims with constant rate r [11, Ch. 8]:

$$V_t = e^{-r\tau}\,\mathbb{E}_t^{\mathbb{Q}}[\Psi]. \tag{2.1}$$

Here, V_t is the option value at time t, $\tau = T - t$ is time to maturity, Ψ is the terminal payoff, and $\mathbb{E}_t^{\mathbb{Q}}[\cdot]$ denotes conditional expectation under the risk-neutral measure $\mathbb{Q}$. The risk-neutral measure is the pricing probability measure under which discounted tradable assets have no expected excess return; this is why future payoffs can be discounted at the risk-free rate r in this formula.

In the following sections, this framework is connected to three ingredients used throughout the thesis: the stochastic-calculus tools needed for the model equations, the Heston stochastic-volatility dynamics, and the characteristic-function/Fourier-COS pricing route.

2.3. Stochastic Calculus Background

This section introduces only the stochastic-calculus notions needed by the pricing methods used later. The presentation follows standard probability definitions and the transform-pricing notation adopted in [11, Ch. 1] and [4, Sec. 2].

For a real random variable X, the cumulative distribution function is

$$F_X(x) = \mathbb{P}(X \leq x). \tag{2.2}$$

Here, x is a real evaluation point and $\mathbb{P}$ denotes probability. If F_X is differentiable, the associated probability density function is

$$f_X(x) = \frac{\partial F_X(x)}{\partial x}. \tag{2.3}$$

Following this notation, the characteristic function of X is

$$\phi_X(u) := \mathbb{E}\left[e^{iuX}\right] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \tag{2.4}$$

for $u \in \mathbb{R}$, where i is the imaginary unit.

Applying the logarithm to ϕ_X, we obtain the cumulant characteristic function

$$\zeta_X(u) = \log \mathbb{E}\left[e^{iuX}\right] = \log \phi_X(u). \tag{2.5}$$

Its Maclaurin expansion is

$$\zeta_X(u) = \sum_{k=0}^{\infty} \left.\frac{\partial^k \zeta_X(u)}{\partial u^k}\right|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \tag{2.6}$$

where $\zeta_k \equiv \left.\frac{\partial^k \zeta_X(u)}{\partial u^k}\right|_{u=0}$ is the k-th cumulant.

The moment generating function is

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}\left[e^{uX}\right]. \tag{2.7}$$

Using $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, cumulants can be computed from the characteristic function as

$$\zeta_k = \left.\frac{\partial^k \zeta_X(u)}{\partial u^k}\right|_{u=0} = \left.\frac{\partial^k}{\partial u^k} \log \phi_X(-iu)\right|_{u=0}. \tag{2.8}$$

These objects are central in the rest of this chapter: the characteristic function provides direct access to Fourier-domain pricing formulas, and low-order cumulants are later used to construct truncation intervals for the COS method.

2.4. The Heston Stochastic Volatility Model

In a general diffusion form, the Heston dynamics can be written as

$$dS_t = \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(S)}, \tag{2.9}$$

$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(v)}, \tag{2.10}$$

$$d\langle W^{(S)}, W^{(v)}\rangle_t = \rho\,dt. \tag{2.11}$$

Here, v_t is the instantaneous variance, $W_t^{(S)}$ and $W_t^{(v)}$ are Brownian motions driving the price and variance shocks, and $d\langle W^{(S)}, W^{(v)}\rangle_t$ denotes their instantaneous quadratic co-variation. The parameter μ denotes a generic drift under the original measure. In the no-dividend risk-neutral pricing form used in this thesis, the drift becomes r:

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(S,\mathbb{Q})}. \tag{2.12}$$

This is the form used in the pricing formulas of the following sections [2], [11].

Here, v_t is the instantaneous variance (equivalent to the v_t notation used in later data-driven chapters). The variance process is of CIR type, so it is mean-reverting toward $\bar{v}$ at speed κ, with volatility of variance controlled by γ.

The five Heston parameters have distinct financial roles. The pair $(\bar{v}, v_0)$ controls the level of variance, with v_0 mainly affecting the short end and $\bar{v}$ anchoring longer maturities. The speed κ determines how quickly variance reverts toward its long-run level. The parameter γ controls the variability of the variance process itself, and therefore the curvature of option smiles. Finally, ρ links spot and variance shocks; in equity-index applications it is usually negative, which generates the left-skewed implied-volatility patterns observed in markets.

This parameter interpretation is directly linked to the implied-volatility (IV) surface, i.e., the map of implied volatilities across strikes and maturities. At a fixed maturity, the cross-section of this surface is often called the volatility smile:

TABLE 2.2. INTERPRETATION OF HESTON PARAMETERS

Parameter	Interpretation	Main effect on IV surface
ρ	Instantaneous correlation between spot and variance shocks.	Controls skew: more negative ρ typically implies stronger left skew.
γ	Volatility of variance (*vol-of-vol*).	Controls curvature and short-maturity smile amplitude; larger γ usually increases convexity.
$\bar{v}$	Long-run mean level of the variance process.	Sets the global IV level around which the surface oscillates across maturities.
v_0	Initial variance level at valuation time (v_{t_0}).	Anchors short-maturity ATM IV and strongly affects the front end of the term structure.
κ	Speed of mean reversion of v_t toward $\bar{v}$.	Controls how fast maturities converge to long-run variance, affecting term-structure smoothing.

Source: Based on [2] and [12, Chs. 2–3]

In this thesis, implied volatility is defined as the value σ_{imp} that reproduces a given option price when inserted into the Black–Scholes formula with the same contract and market inputs [1]:

$$V_{BS}(S_0, K, r, \tau, \sigma_{imp}) = V_{\text{target}}. \tag{2.13}$$

Here, S_0 is the initial value of the underlying price process, V_{BS} is the Black–Scholes price, and V_{target} is the target option value to be matched. Therefore, implied volatility is a quoting transformation of prices (not a structural parameter of Heston). It is nevertheless the natural comparison space for calibration and machine-learning experiments, because market option quotes are usually summarized across strikes and maturities through IV surfaces rather than raw prices. The numerical inversion used to compute σ_{imp} is detailed in Section 2.7.

A common positivity condition for the CIR variance process is the Feller condition

$$2\kappa\bar{v} \geq \gamma^2. \tag{2.14}$$

In practice, calibrated parameter sets may violate this inequality while still producing usable market fits; therefore, it is treated as a diagnostic and stability indicator rather than as a hard calibration constraint in many numerical workflows [12, Chs. 2–3].

For pricing, the key advantage of Heston is that the log-price characteristic function is available in closed (semi-analytical) form [2]. This makes the model especially suitable for Fourier-domain valuation, which is the computational route adopted in the next sections.

2.5. Pricing Via Characteristic Functions

When the characteristic function is available in closed or semi-closed form, pricing in the Fourier domain becomes a natural alternative to direct density integration. Following [4, Sec. 2], we write the option value in terms of the conditional transition density of the terminal state variable.

Using the notation adopted in the next section, with $x = X(t_0)$ and $y = X(T)$, the discounted valuation formula is

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y)\, f_X(T, y; t_0, x)\, dy. \tag{2.15}$$

Here, t_0 is the valuation date, X is the state variable used for transform pricing, $V(T, y)$ is the terminal payoff as a function of the terminal state y, and $f_X(T, y; t_0, x)$ is the conditional transition density of $X(T) = y$ given $X(t_0) = x$. The corresponding conditional characteristic function is

$$\phi_X(u; t_0, x) := \mathbb{E}^{\mathbb{Q}}_{t_0}\Big[e^{iuX(T)} \mid X(t_0) = x\Big] = \int_{\mathbb{R}} e^{iuy} f_X(T, y; t_0, x)\, dy. \tag{2.16}$$

Under standard integrability conditions, the density can be recovered by Fourier inversion:

$$f_X(T, y; t_0, x) = \frac{1}{2\pi} \int_{\mathbb{R}} e^{-iuy} \phi_X(u; t_0, x)\, du. \tag{2.17}$$

Let $\hat{V}_T(u) := \int_{\mathbb{R}} e^{-iuy} V(T, y)\, dy$ denote the Fourier transform of the payoff at maturity. Substituting the inverse representation of f_X into the pricing integral and exchanging the order of integration gives

$$V(t_0, x) = e^{-r\tau} \frac{1}{2\pi} \int_{\mathbb{R}} \phi_X(u; t_0, x)\, \hat{V}_T(u)\, du. \tag{2.18}$$

Compared with PDE solvers, this approach avoids full space-time gridding. Compared with Monte Carlo, it is deterministic and does not introduce sampling noise. In repeated-pricing tasks (surface generation, calibration loops, and large synthetic datasets), this is a major practical advantage.

The Fourier-COS method used in the next section is a specific spectral discretization of this transform-based pricing framework: it replaces the continuous integral by a finite cosine expansion over a truncated interval, preserving high accuracy with efficient vectorized evaluation.

2.6. The Fourier-COS method

The Fourier-COS method approximates transition densities and option values by a cosine series on a finite interval [4, Secs. 3–4]. Its practical appeal in this thesis is that it combines spectral convergence with efficient vectorized evaluation once the model characteristic function is available.

Let $x = X(t_0)$ and $y = X(T)$, and consider a truncation interval $[a, b]$ that concentrates most of the probability mass of $X(T) \mid X(t_0) = x$. On this interval, the conditional density is approximated by

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \tag{2.19}$$

where $\hat{f}_X$ is the truncated cosine approximation of the density, N is the number of cosine terms, k indexes the expansion terms, $\sum'$ indicates that the $k = 0$ term is weighted by $1/2$, and the cosine coefficients are obtained from the characteristic function:

$$\bar{F}_k := \frac{2}{b-a} \Re\left[\phi_X\left(\frac{\pi k}{b-a}\right) \exp\left(-\frac{i\pi a k}{b-a}\right)\right]. \tag{2.20}$$

The operator $\Re[\cdot]$ denotes the real part.

Starting from

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x)\, dy, \tag{2.21}$$

the COS discretization yields

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1}{}' \Re\left[\phi_X\left(\frac{\pi k}{b-a}\right) \exp\left(-\frac{i\pi a k}{b-a}\right)\right] H_k, \tag{2.22}$$

where the payoff coefficients are

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos\left(\frac{y-a}{b-a}\pi k\right) dy. \tag{2.23}$$

For plain-vanilla options, H_k has closed-form expressions, so the numerical effort is concentrated in evaluating the characteristic function on a fixed frequency grid [4, Sec. 3].

The choice of $[a, b]$ controls truncation error. Following the cumulant-based rule in [4, Sec. 5], we use

$$[a, b] = \left[c_1 - L\sqrt{c_2 + \sqrt{c_4}},\ c_1 + L\sqrt{c_2 + \sqrt{c_4}}\right], \tag{2.24}$$

with c_1, c_2, c_4 the first, second, and fourth cumulants of $X(T)$ and $L > 0$ a scaling parameter. This connects directly with the cumulant machinery introduced in Section 2.3. In the numerical experiments of this thesis, the integration interval is selected using the refined cumulant-based strategy detailed in Appendix A.

From a numerical viewpoint, it is useful to separate two error sources:

$$\left|V - V_N^{\mathrm{COS}}\right| \leq \underbrace{\varepsilon_{\mathrm{trunc}}([a,b])}_{\text{domain truncation}} + \underbrace{\varepsilon_N}_{\text{finite cosine series}} . \tag{2.25}$$

Here, V_N^{COS} is the COS approximation using N terms, $\varepsilon_{\mathrm{trunc}}$ is the error from truncating the integration domain to $[a,b]$, and ε_N is the finite-series error. Increasing L reduces $\varepsilon_{\mathrm{trunc}}$ but may require larger N to keep ε_N small; reducing L has the opposite effect. In practice, we choose (L,N) jointly and verify stability, especially for extreme strikes and short maturities, where numerical artifacts appear first.

Implementation is fully vectorized: frequencies $u_k = \pi k/(b-a)$, exponential phase factors, and payoff coefficients are precomputed once per maturity, then reused across strike grids. This is one of the reasons COS is well suited for repeated-pricing workflows such as calibration loops and synthetic dataset generation.

2.6.1. The COS Method Applied To The Heston Model

Following the notation of Sections 2.5–2.6 and [4, Sec. 3], define

$$u_k := \frac{k\pi}{b-a}, \qquad x := \log\left(\frac{S_0}{K}\right), \qquad y := \log\left(\frac{S_T}{K}\right).$$

Here, x and y are log-moneyness variables: they measure the underlying price relative to the strike before and at maturity. Positive log-moneyness means the underlying is above the strike, while negative log-moneyness means it is below the strike. Under Heston dynamics, the conditional characteristic function of y can be written in affine form [2], [12]

$$\varphi_H(u;\tau,x,v_0) = \exp(C(u,\tau) + D(u,\tau)v_0 + iux), \tag{2.26}$$

with

$$d(u) = \sqrt{(\kappa - i\rho\gamma u)^2 + \gamma^2\,(u^2 + iu)}, \tag{2.27}$$

$$g(u) = \frac{\kappa - i\rho\gamma u - d(u)}{\kappa - i\rho\gamma u + d(u)}, \tag{2.28}$$

$$C(u,\tau) = iur\tau + \frac{\kappa\bar{v}}{\gamma^2}\left[(\kappa - i\rho\gamma u - d(u))\,\tau - 2\log\left(\frac{1-g(u)e^{-d(u)\tau}}{1-g(u)}\right)\right], \tag{2.29}$$

$$D(u,\tau) = \frac{\kappa - i\rho\gamma u - d(u)}{\gamma^2}\left(\frac{1-e^{-d(u)\tau}}{1-g(u)e^{-d(u)\tau}}\right). \tag{2.30}$$

Here, φ_H denotes the Heston characteristic function of the log-moneyness state, while C, D, d, and g are complex-valued auxiliary functions of the Fourier variable u and maturity τ. This is the standard semi-closed representation used in Fourier pricing; numerically, consistency in branch selection for $d(u)$ and the complex logarithm is required for stability [12, Chs. 2–3].

The COS price for one strike can then be written as

$$V(t_0, x) \approx K e^{-r\tau} \sum_{k=0}^{N-1}{}' \Re\Big[\varphi_H(u_k; \tau, x, v_0)\, e^{-iu_k a}\Big]\, U_k, \tag{2.31}$$

where U_k are payoff-dependent COS coefficients. For plain-vanilla options:

$$U_k^{\text{call}} = \frac{2}{b-a}\left(\chi_k(0, b) - \psi_k(0, b)\right), \qquad U_k^{\text{put}} = \frac{2}{b-a}\left(-\chi_k(a, 0) + \psi_k(a, 0)\right), \tag{2.32}$$

with

$$\chi_k(c, d) = \frac{1}{1+u_k^2}\Big[e^d(\cos(u_k(d-a)) + u_k \sin(u_k(d-a))) - e^c(\cos(u_k(c-a)) + u_k \sin(u_k(c-a)))\Big], \tag{2.33}$$

$$\psi_k(c, d) = \begin{cases} d - c, & k = 0, \\ \dfrac{b-a}{k\pi}\left[\sin(u_k(d-a)) - \sin(u_k(c-a))\right], & k \geq 1. \end{cases} \tag{2.34}$$

Here, χ_k and ψ_k are analytical payoff integrals over generic bounds c and d inside the truncation interval. In the implementation of this thesis, data generation is run on European puts as a numerical convention, mainly associated with the COS-pricing setup used in the project configuration.

This subsection establishes the analytical Heston-COS pricing block used as the reference framework in the following chapters.

2.6.2. Semi-Analytical Greeks from the Heston Characteristic Function

For validation purposes, Greeks can also be defined in the semi-analytical Heston framework, i.e., directly from the characteristic-function pricing representation. For a European call, a standard form is [2] and is developed in [12, Chapter 1]

$$C(S_0, K, \tau) = S_0\, \Pi_1 - K e^{-r\tau}\, \Pi_2, \tag{2.35}$$

where $C(S_0, K, \tau)$ is the European call price and Π_1, Π_2 are risk-neutral probability-type terms in the Heston pricing formula:

$$\Pi_j = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Re\left(\frac{e^{-iu\log K}\, \Psi_j(u)}{iu}\right) du, \qquad j \in \{1, 2\}, \tag{2.36}$$

with

$$\Psi_2(u) = \phi_X(u), \qquad \Psi_1(u) = \frac{\phi_X(u-i)}{\phi_X(-i)}, \tag{2.37}$$

where j selects the two probability terms, Ψ_j are the corresponding Fourier integrands, and ϕ_X is the characteristic function of $X_T = \log S_T$ under Heston [2]; see also [12, Chapter 1].

In this context, Greeks are sensitivities of the option value to market or model inputs. Delta measures sensitivity to the underlying price, Gamma measures the curvature of that sensitivity, Vega measures sensitivity to volatility or variance, Rho measures sensitivity to the interest rate, and Theta measures sensitivity to the passage of time.

Let p denote a generic model/input variable ($r, \tau, v_0, \rho, \kappa, \gamma, \bar{v}$, etc.). Under standard regularity and integrability conditions for affine models, differentiation under the integral sign yields [12, Chapter 11]

$$\partial_p \Pi_j = \frac{1}{\pi} \int_0^\infty \Re\left(\frac{e^{-iu \log K}}{iu} \partial_p \Psi_j(u) \right) du. \tag{2.38}$$

Therefore, Greeks are obtained by combining the derivatives of Π_1, Π_2 through the pricing identity [12, Chapter 11]. For example:

$$\Delta_{\text{call}} = \partial_{S_0} C = \Pi_1 + S_0 \, \partial_{S_0} \Pi_1 - K e^{-r\tau} \partial_{S_0} \Pi_2, \tag{2.39}$$

$$\Gamma_{\text{call}} = \partial_{S_0}^2 C = 2\partial_{S_0} \Pi_1 + S_0 \partial_{S_0}^2 \Pi_1 - K e^{-r\tau} \partial_{S_0}^2 \Pi_2, \tag{2.40}$$

$$\text{Vega}_{v_0} = \partial_{v_0} C = S_0 \partial_{v_0} \Pi_1 - K e^{-r\tau} \partial_{v_0} \Pi_2, \tag{2.41}$$

$$\text{Rho} = \partial_r C = S_0 \partial_r \Pi_1 + K\tau e^{-r\tau} \Pi_2 - K e^{-r\tau} \partial_r \Pi_2, \tag{2.42}$$

$$\Theta = \partial_t C = -\partial_\tau C. \tag{2.43}$$

The practical advantage is that all sensitivities are expressed in terms of derivatives of the same characteristic-function integrands. In affine form, $\phi_X(u) = \exp(C(u, \tau) + D(u, \tau) v_0 + iu \log S_0)$, so parameter derivatives follow analytically from derivatives of C and D, and can be inserted directly into the integrals above.

2.7. Implied Volatility And Numerical Inversion

For each target option value V_{target} (market quote or model-generated price), implied volatility is obtained by inverting the Black–Scholes map [1]. This provides a common representation across strikes and maturities, which is the main comparison space used in this thesis.

Numerically, we solve the scalar nonlinear equation

$$F(\sigma) := V_{BS}(S_0, K, r, \tau, \sigma) - V_{\text{target}} = 0, \qquad \sigma > 0. \tag{2.44}$$

For European vanilla options, V_{BS} is strictly increasing in σ when $\tau > 0$, so the root is unique whenever V_{target} satisfies static no-arbitrage bounds.

The inversion routine used in this thesis is the bracketed Brent method [13, Ch. 9]. This choice is robust for the one-dimensional IV root and avoids derivative-sensitivity issues in low-vega regions. As an alternative, Levenberg–Marquardt was also tested in preliminary experiments, but it showed lower numerical stability in this workflow; therefore, it is not used in the final pipeline.

Stopping is based on standard price and parameter tolerances:

$$|F(\sigma_n)| \leq \varepsilon_{\text{price}} \quad \text{or} \quad |\sigma_{n+1} - \sigma_n| \leq \varepsilon_\sigma, \qquad n \leq n_{\max}. \tag{2.45}$$

Here, σ_n is the volatility iterate at step n, $\varepsilon_{\text{price}}$ and ε_σ are stopping tolerances, and $n_{\max}$ is the maximum number of iterations. If convergence is not reached, or if the target price violates arbitrage bounds, the point is flagged as invalid for IV reporting. In edge cases near the IV floor, the pricing stage may be recomputed with a wider COS truncation interval before retrying inversion.

In the results chapters, IV errors are the primary evaluation metrics:

$$\text{MAE}_{IV} = \frac{1}{N} \sum_{i=1}^{N} \left| \hat{\sigma}_i - \sigma_i^{\text{ref}} \right|, \qquad \text{RMSE}_{IV} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(\hat{\sigma}_i - \sigma_i^{\text{ref}} \right)^2}. \tag{2.46}$$

Here, MAE_{IV} and RMSE_{IV} are the mean absolute error and root mean squared error in IV units, $\hat{\sigma}_i$ is the model-implied volatility prediction, σ_i^{ref} is the reference implied volatility, and N is the number of evaluation points in the metric. To capture tail behavior, quantiles of absolute IV error are also reported (e.g., q_{95} and q_{99}), together with stratified summaries by moneyness and maturity buckets.

3. NEURAL NETWORK PRICER FOR HESTON

The main objective of this chapter is to construct an ANN-based surrogate pricer for European vanilla options under the Heston stochastic-volatility model. The target quantity is implied volatility, and the surrogate is designed to approximate the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

Reference pricing based on numerical methods (in our pipeline, Heston COS valuation plus implied-volatility inversion) is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. The ANN aims to preserve pricing accuracy while reducing inference time by several orders of magnitude.

Methodologically, this chapter follows the ANN-pricing framework of Liu et al. [5] as the main reference (architecture family, data ranges, and training philosophy), while explicitly documenting the implementation choices adopted in this project.

This chapter focuses exclusively on the pricing network; the calibration architecture is treated separately in Chapter 4.

3.1. Problem Formulation

In this work, vanilla option pricing under Heston is posed as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \to \mathbb{R}, \qquad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where $m = S_0/K$ is moneyness and $\tau = T - t$ is time-to-maturity. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In this chapter, we use the ANN only as a pricing surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogate is primarily an in-domain approximator. Preliminary out-of-domain checks indicate that the network remains stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Data Formulation

Training requires a large set of labeled input-output pairs. To obtain a controlled and fully reproducible dataset, we use synthetic data generated under the Heston model. For each sampled input vector, the target implied volatility is computed through the numerical pricing pipeline (COS valuation followed by implied-volatility inversion).

In the current pipeline, data generation is organized as a *master dataset* shared across downstream models. Each row stores:

- features: $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$,
- targets: $(price_{\cos}, \sigma_{\text{imp}}^{\text{brent}})$,
- quality labels: $(feller\_slack, feller\_ok, brent\_success, brent\_abs\_residual)$.

This design allows ANN and PINN workflows to reuse the same synthetic source while applying experiment-specific filters only at training time.

The raw variable set includes Heston parameters $\Theta = \{\rho, \kappa, \gamma, \bar{v}, v_0\}$ and contract/market variables $\Sigma = \{K, S_0, \tau, r\}$, which defines a 9-dimensional space. In our setup, we fix $K = 1$ and represent the underlying level through moneyness $m = S_0/K$, leading to an 8-dimensional input vector:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^8 (already intractable for moderate N). Therefore, the synthetic inputs are sampled with Latin Hypercube Sampling (LHS), which stratifies each coordinate and provides broad marginal coverage with a fixed budget [14]. In our implementation, LHS is applied directly to the full input vector with a fixed seed and a target size of 10^7 samples in the master dataset. The specific ranges, constraints, and splitting/preprocessing rules are detailed below.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [5]), the training domain is summarized in Table 3.1.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9,\ 0.0]$
κ	$[0.0,\ 3.0]$
γ	$[0.01,\ 0.8]$
$\bar{v}$	$[0.01,\ 0.5]$
v_0	$[0.05,\ 0.5]$
m	$[0.6,\ 1.4]$
τ	$[0.05,\ 3.0]$
r	$[0.0,\ 0.05]$

Source: Based on [5] and project configuration choices

To avoid non-physical or numerically unstable samples, we enforce $\tau > 0$ and monitor a Feller-type admissibility condition

$$2\kappa\bar{v} - \gamma^2 \geq 0,$$

through the scalar label

$$feller\_slack = 2\kappa\bar{v} - \gamma^2,$$

and the binary indicator $feller\_ok = \mathbb{1}\{feller\_slack \geq 0\}$. Importantly, Feller-violating points are *not* removed during generation; they are kept in the master dataset for controlled ablation studies.

During synthesis we fix the strike to $K = 1$ and price European put options. Fixing K is equivalent to expressing spot through moneyness, which reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Dataset Splitting And Preprocessing

After generation, we keep all selected rows in the master dataset and split into train/-validation/test subsets with proportions 0.8/0.1/0.1. In the reported configuration, splits are random (without mandatory stratification), and filtering policies are deferred to each training experiment.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline through explicit labels, not hard row deletion by default. In particular, we track Brent inversion status and reconstruction error

$$|V_{BS}(\sigma_{imp}) - V_{\text{target}}|,$$

while keeping all generated points available for later filtering. For storage efficiency, output tables are persisted in `float32`, whereas COS pricing and Brent inversion are computed internally in `float64`.

3.2.3. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps. First, the target option value V_{target} is computed with the Heston COS solver for a given input vector $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$. Second, implied volatility is obtained by solving

$$V_{BS}(\sigma) = V_{\text{target}}$$

with a Brent root-finding routine under fixed bounds and tolerance [13].

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8}, bounded search interval, and iteration cap). Additionally, when the inferred volatility hits the lower IV floor, an adaptive COS fallback can be triggered: the truncation width is increased ($L \leftarrow 1.5L$, with optional N scaling) and inversion is retried. The retry is accepted only if it improves numerical quality (e.g., leaves the floor or reduces reconstruction error).

Rather than dropping those edge cases at generation time, the pipeline records quality metadata (`brent_success`, `brent_abs_residual`, and retry diagnostics) so that robustness filters can be applied transparently at training time. This preserves full traceability of numerical reliability across the domain.

3.3. Neural Network Architecture

The pricing surrogate is implemented as a fully connected neural network that approximates a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

This choice is also consistent with the classical universal-approximation results: under standard assumptions, feed-forward networks with non-polynomial activations can approximate continuous functions on compact sets to arbitrary accuracy [15], [16]. In this thesis, this theorem is used as a representation-capacity motivation, while practical accuracy is established empirically through out-of-sample benchmarks.

3.3.1. Model Class

Following [5], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The architecture used in this chapter is

$$8 \to 200 \to 200 \to 200 \to 200 \to 1,$$

with ReLU activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \ldots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \max(0, \cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	Value
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	ReLU
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [5]

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training is fully configuration-driven and follows an epoch-based supervised learning loop. At each epoch, the active optimizer performs parameter updates on the training split, then the model is evaluated on the validation split without gradient computation. For Adam updates, optimization is done with standard mini-batch backpropagation; for L-BFGS updates, each step is computed through a closure that evaluates the loss and its gradient on the selected batch regime.

Each epoch stores train/validation metrics, current optimizer identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (`mps` when available, otherwise `cpu`), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^{B}$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\mathrm{MSE}}(\mathbf{w}) = \frac{1}{B}\sum_{i=1}^{B}(\hat{\sigma}_i - \sigma_i)^2 .$$

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. Although MAE and RMSE are available in the implementation, MSE is used as the main training criterion in the reported baseline.

3.4.2. Optimizer And Hyperparameters

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Several optimizer configurations were tested (Adam, mixed schedules, and `mix_half`). For model selection, we prioritize out-of-sample absolute-error behaviour (MAE and quantile stability), since these metrics better reflect the typical pointwise error level used later in calibration and sensitivity tasks.

Under this criterion, the selected reference ANN is `Liu_like_v01` (Adam), while `MIX_half_v01` is the second-best candidate. Although `MIX_half_v01` achieves a slightly lower validation MSE, `Liu_like_v01` provides lower validation and test MAE, together with better high-quantile absolute errors.

TABLE 3.3. REFERENCE AND RUNNER-UP RUNS USED IN OPTIMIZER SELECTION

Run	Optimizer mode	Val MAE	Test MAE
`Liu_like_v01`	Adam	4.1132×10^{-4}	4.2925×10^{-4}
`MIX_half_v01`	Adam $\rightarrow$ L-BFGS	5.1342×10^{-4}	5.2377×10^{-4}

Source: Own elaboration from `outputs/runs/*/metrics/eval_global.csv`

The hyperparameters of the selected reference run (`Liu_like_v01`) are summarized in Table 3.4.

TABLE 3.4. TRAINING HYPERPARAMETERS OF THE SELECTED RUN (`LIU_LIKE_V01`)

Hyperparameter	Value
Training mode	Adam
Seed (data-loader generator)	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Loss	MSE
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, step_size = 500, $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.4.3. Regularization And Callbacks

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Regularization in the baseline is intentionally light: dropout is disabled ($p = 0$), batch normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of optimizer scheduling and data quality controls.

Callback management includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration improves training robustness without over-constraining the optimization path: checkpoints protect against late-epoch degradation, while the Adam-to-L-BFGS transition plus scheduler supports stable convergence. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

4. ANN-ACCELERATED CALIBRATION OF THE HESTON MODEL

This chapter develops the calibration stage used in the thesis pipeline. The objective is to recover a market-consistent Heston parameter vector

$$\theta = (\rho, \kappa, \gamma, \bar{v}, v_0)$$

from an implied-volatility surface. The chapter is primarily based on the neural calibration framework of Liu et al. [5]. In that approach, calibration is not performed by repeatedly calling the full Heston pricing engine. Instead, a trained neural pricer is kept fixed and used as a fast forward map inside an inverse optimization loop.

The role of this chapter is therefore methodological. The contribution is not a new calibration architecture in isolation, but a reproducible implementation and adaptation of the Liu et al. workflow within the present thesis pipeline, with explicit diagnostics for residual fit, parameter recovery, regularization, and local identifiability. The quantitative benchmark results are reported later in Chapter 6; here we define the calibration problem and explain the design choices that make the benchmark interpretable.

To make the dependence on [5] explicit, the main inherited ideas are:

- using an ANN pricing surrogate to accelerate repeated Heston evaluations during calibration;
- formulating calibration in implied-volatility space;
- treating the calibrated Heston vector $(\rho, \kappa, \gamma, \bar{v}, v_0)$ as the inverse-problem output;
- solving the inverse problem with a global optimizer rather than training a separate inverse network in this implementation.

The additions in this thesis are the integration with the reproducible COS/Brent data pipeline, the connection with the downstream PINN stage, and the local smoothness/curvature diagnostics used to interpret identifiability around the calibrated optimum.

4.1. Role of Calibration in the Pipeline

The supervised ANN pricer of Chapter 3 approximates the forward map

$$f_{NN} : (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{\mathrm{IV}},$$

where σ_{IV} denotes implied volatility. Calibration uses the same map in the inverse direction: given a set of observed implied volatilities over moneyness and maturity, it searches for the Heston parameters that reproduce those observations as closely as possible. This forward-surrogate-in-the-loop view is the central CaNN idea adopted from Liu et al. [5].

This stage is the bridge between the synthetic pricing surrogate and the downstream PINN analysis. Once the calibrated vector $\theta^\star$ is obtained, it defines the market state consumed by later pricing and Greek-computation experiments. In particular, Chapter 5 uses the calibrated structural parameters $(\rho^\star, \kappa^\star, \gamma^\star, \bar{v}^\star)$ and the calibrated initial variance $v_0^\star$ to evaluate the PINN at the corresponding market configuration.

Placeholder: CaNN workflow graph
Market IV quotes $\rightarrow$ quote grid $(m, \tau, r, \sigma_{\mathrm{IV}}, w)$ $\rightarrow$ candidate Heston parameters θ $\rightarrow$ frozen ANN pricer $\rightarrow$ predicted IV surface $\rightarrow$ weighted calibration objective $\rightarrow$ Differential Evolution $\rightarrow$ calibrated vector $\theta^\star$ $\rightarrow$ PINN/pricing/Greeks stage.

Fig. 4.1. Conceptual graph of the ANN-accelerated calibration workflow. This figure should later be replaced by a clean pipeline diagram.

The term CaNN is kept as a convenient shorthand for the calibration stage of [5]. In the present implementation, however, the calibrated parameters are not produced by training a separate supervised inverse network. The central object is a fixed forward ANN pricer embedded in a numerical optimizer.

4.2. Inverse Calibration Problem

Following the notation and calibration logic of [5], let the calibration surface contain N_q market quotes

$$Q = \{(m_j, \tau_j, r_j, \sigma_j^{\mathrm{mkt}}, w_j)\}_{j=1}^{N_q},$$

where $m_j = S_0/K_j$ is moneyness, τ_j is time to maturity, r_j is the risk-free rate, σ_j^{mkt} is the observed implied volatility, and $w_j \geq 0$ is an optional quote weight.

For a candidate Heston vector θ, the ANN pricer predicts the implied-volatility surface

$$\hat{\sigma}_j(\theta) = f_{NN}(\rho, \kappa, \gamma, \bar{v}, v_0, m_j, \tau_j, r_j), \qquad j = 1, \ldots, N_q.$$

The calibration residual at quote j is

$$e_j(\theta) = \hat{\sigma}_j(\theta) - \sigma_j^{\mathrm{mkt}}.$$

The objective minimized in calibration is the weighted implied-volatility mismatch, optionally augmented with a regularization term:

$$J(\theta) = \frac{1}{\sum_{j=1}^{N_q} w_j} \sum_{j=1}^{N_q} w_j e_j(\theta)^2 + \lambda R(\theta). \tag{4.1}$$

In the implementation, the optimizer may use the unnormalized weighted sum instead of the normalized weighted mean; this changes the objective by a positive constant factor when $\lambda = 0$, and therefore does not change the minimizer. Reported benchmark metrics use the normalized weighted-MSE convention.

The calibrated vector is

$$\theta^\star = \arg\min_{\theta \in \mathcal{B}} J(\theta), \tag{4.2}$$

where $\mathcal{B}$ denotes the admissible parameter box. In the reported experiments, the order of the calibrated parameters is

$$(\rho, \kappa, \gamma, \bar{v}, v_0).$$

TABLE 4.1. CALIBRATION PARAMETER BOUNDS USED IN THE CANN STAGE.

Parameter	**Lower bound**	**Upper bound**
ρ	−0.9	0.0
κ	0.0	3.0
γ	0.01	0.8
$\bar{v}$	0.01	0.5
v_0	0.05	0.5

Source: Own configuration, aligned with the ANN training domain.

The bounds serve two purposes. First, they encode the parameter region in which the Heston model is considered admissible for the experiments. Second, they keep calibration inside the interpolation domain of the ANN pricer, limiting extrapolation risk during the inverse search.

4.3. Input Representation: Implied-Volatility Surfaces

Calibration is performed in implied-volatility space rather than in price space, as in Liu et al. [5]. This choice is consistent with the training target of the ANN pricer and with the market convention of comparing option surfaces across strikes and maturities. It also reduces scale effects: raw option prices vary strongly with moneyness, maturity, discounting, and payoff level, while implied volatility provides a more homogeneous representation of the smile surface.

Each calibration input is represented as a table of quotes. At minimum, the table contains the columns shown in Table 4.2.

TABLE 4.2. QUOTE-TABLE REPRESENTATION USED IN CALIBRATION.

Column	Meaning
`moneyness`	$m = S_0/K$, using the same convention as the ANN pricer.
`tau`	Time to maturity $\tau = T - t$.
`r`	Risk-free rate associated with the quote.
`iv_market`	Observed or synthetic market implied volatility.
`weight`	Optional quote weight used in the objective. If absent, unit weights are used.

Source: Own calibration pipeline.

For a fixed candidate θ, the calibration pipeline constructs the ANN feature matrix

$$X(\theta) = \begin{bmatrix} \rho & \kappa & \gamma & \bar{v} & v_0 & m_1 & \tau_1 & r_1 \\ \rho & \kappa & \gamma & \bar{v} & v_0 & m_2 & \tau_2 & r_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \rho & \kappa & \gamma & \bar{v} & v_0 & m_{N_q} & \tau_{N_q} & r_{N_q} \end{bmatrix}.$$

The Heston parameters are repeated across all quotes, while (m_j, τ_j, r_j) vary by contract. A single batched ANN forward pass then produces the full predicted surface for that candidate.

Placeholder: quote surface representation
Suggested plot: observed IV surface over the (m, τ) grid, with moneyness on the horizontal axis, maturity on the vertical axis, and color indicating $\sigma_{\mathrm{IV}}^{\mathrm{mkt}}$. Candidate source: `data/market/market_quotes_liu_35.csv`.

Fig. 4.2. Placeholder for the implied-volatility surface used as calibration input.

4.4. Architecture of the Calibration Stage

The calibration architecture follows the Liu et al. principle of separating the expensive forward pricing approximation from the inverse optimization task [5]. In this implementation it is a composition of three components:

1. a frozen ANN pricer trained in Chapter 3,
2. a quote-table encoder that builds ANN inputs for each candidate parameter vector,
3. a global optimizer that updates candidate parameter vectors to reduce the calibration objective.

This is different from training a direct inverse map from a volatility surface to Heston parameters. Direct inverse networks can be extremely fast at inference time, but they require a dedicated supervised dataset of surfaces and known parameters. In line with the optimization-based calibration route in [5], the calibration stage in this thesis solves the inverse problem explicitly for the observed quote set. This makes the objective value, residual map, and optimizer diagnostics directly inspectable.

For a population of S candidate vectors, the implementation vectorizes the evaluation. If

$$\Theta_{\text{pop}} = \{\theta^{(1)}, \ldots, \theta^{(S)}\},$$

then the feature matrix contains $S \times N_q$ rows. The ANN returns $S \times N_q$ predicted volatilities, which are reshaped into S surfaces and scored by Equation (4.1). This vectorized design is essential because Differential Evolution evaluates many candidates per iteration.

Placeholder: vectorized CaNN evaluation diagram
Candidate population ($5 \times S$) plus quote grid (N_q) $\rightarrow$ batched ANN input ($S N_q \times 8$) $\rightarrow$ predicted surfaces ($S \times N_q$) $\rightarrow$ objective values (S).
This should be a graph-style figure, not a numerical plot.

Fig. 4.3. Placeholder for the batched evaluation architecture used by the CaNN stage.

4.5. Differential Evolution Optimization

The inverse calibration surface can be non-convex, especially when several parameter combinations generate similar implied-volatility smiles. For this reason, the optimizer used in the baseline calibration is Differential Evolution (DE), a population-based global optimization method for continuous variables [17]. This choice follows the calibration philosophy in [5]: the ANN makes objective evaluations cheap enough that a global population search becomes practical.

DE maintains a population of candidate parameter vectors inside the admissible bounds. At each iteration, new trial vectors are generated through mutation and crossover operations. A trial vector replaces an existing candidate if it improves the objective. This selection rule gradually moves the population toward lower-residual regions without requiring gradients of the objective.

The main DE controls used in the implementation are summarized in Table 4.3.

TABLE 4.3. DIFFERENTIAL EVOLUTION CONTROLS USED IN THE CALIBRATION PIPELINE.

Control	**Value / interpretation**
Maximum iterations	1000
Population size multiplier	20, giving an effective population proportional to the number of calibrated parameters.
Tolerance	10^{-4}
Mutation range	[0.5, 1.0]
Recombination	0.7
Polish	Enabled, so the final DE solution can be locally refined.

Source: Own calibration configuration.

The main advantage of DE in this setting is robustness: it does not depend on a single initial point and can explore several basins of attraction. The cost is that it requires many objective evaluations. This cost would be prohibitive with a full numerical Heston solver in the inner loop, but it is manageable when each surface evaluation is reduced to a batched ANN inference call.

Placeholder: optimization trace

Suggested plot: best objective value versus DE iteration, optionally with median population objective.

Useful to show convergence speed and whether the population stagnates before polish.

Fig. 4.4. Placeholder for the Differential Evolution convergence trace.

4.6. Synthetic Calibration Experiment

The benchmarked calibration experiment uses a synthetic quote set designed to reproduce the Liu-style calibration setting under a controlled ground truth [5]. The quote universe contains 35 contracts arranged as a complete 5×7 grid in moneyness and maturity. The interest rate is constant across quotes in the reference run.

The synthetic setting has two advantages. First, the true parameter vector is known, so calibration can be evaluated both at the surface-fit level and at the parameter-recovery level. Second, the quote set remains reproducible, which allows regularization, optimizer settings, and curvature diagnostics to be compared under identical inputs.

Placeholder: synthetic quote grid
Suggested plot: grid of the 35 calibration quotes in (m, τ), possibly overlaid on the observed IV heatmap.
Candidate source: `data/market/market_quotes_liu_35.csv`.

Fig. 4.5. Placeholder for the synthetic calibration quote universe.

The ground-truth vector is used only for evaluation. It is not passed to the optimizer. During calibration, the optimizer only sees the quote table and the admissible parameter bounds. This distinction is important: surface fit measures whether the model can reproduce the observed implied-volatility quotes, while parameter recovery measures whether the inverse problem identifies the original generating parameters.

The two diagnostics are related but not equivalent. A very small residual does not guarantee exact recovery of every Heston parameter, because different parameter combinations may induce very similar volatility surfaces over a finite set of strikes and maturities. This is one reason why the curvature analysis in Section 4.8 is included.

Placeholder: residual heatmap after calibration
Candidate existing plot:
`figures/cann/cann_residual_heatmap_m_tau_no_reg_smooth.png`.
Purpose: show where the calibrated ANN-implied surface overfits or underfits the quote grid.

Fig. 4.6. Placeholder for the signed residual heatmap $\hat{\sigma}_{\mathrm{IV}} - \sigma_{\mathrm{IV}}^{\mathrm{mkt}}$.

4.7. Regularization and Generalization

The optional regularization term $R(\theta)$ in Equation (4.1) is included to control the inverse problem when the quote set is sparse or the residual surface contains nearly flat directions. The implementation supports no regularization, L^1, L^2, and squared-L^2 penalties:

$$R_{L^1}(\theta) = ||\theta||_1, \qquad R_{L^2}(\theta) = ||\theta||_2, \qquad R_{L^2\text{-sq}}(\theta) = ||\theta||_2^2.$$

In the reported synthetic experiment, the no-regularization run is used as the reference because it gives the best residual fit and the L^2 variant produces only a marginal change in the objective. Nevertheless, keeping the regularization interface is useful for future market-data experiments, where noisy quotes, bid–ask effects, or incomplete surfaces may make unconstrained calibration less stable.

Generalization in this chapter has a specific meaning. The calibration optimizer is not learning a new model; it is searching through the input space of the already trained ANN pricer. Therefore, the main generalization risk is inherited from the forward surrogate. If the optimizer moves into regions poorly represented during ANN training, the calibration objective may become an artifact of extrapolation rather than a meaningful Heston fit. The parameter bounds in Table 4.1 and the quote-domain controls are used to keep calibration within the region where the ANN benchmark in Chapter 6 supports reliable interpolation.

Placeholder: no-regularization vs regularization comparison
Suggested plot: residual RMSE or weighted MSE for $\lambda = 0$ and selected L^2 values; optionally include parameter-error bars.
Candidate existing artifacts:
`outputs/calibration/Liu_like_v01/Calibration_no_reg` and
`outputs/calibration/Liu_like_v01/Calibration_l2_reg`.

Fig. 4.7. Placeholder for the regularization mini-study in the CaNN calibration.

4.8. Smoothness, Hessian Diagnostics, and Identifiability

A calibration result should not be assessed only by its final residual. It is also important to understand the local geometry of the objective around the selected optimum. This is especially relevant in Heston calibration, where some parameters may be weakly identified by a finite implied-volatility surface.

Let

$$e(\theta) = \begin{bmatrix} e_1(\theta) & \cdots & e_{N_q}(\theta) \end{bmatrix}^\top$$

be the residual vector. Ignoring the normalization constant and regularization for readability, the data term can be written as

$$J_{\text{data}}(\theta) = e(\theta)^\top W e(\theta),$$

where $W = \text{diag}(w_1, \ldots, w_{N_q})$. The gradient and Hessian of this objective encode first- and second-order sensitivity of the fit with respect to Heston parameters.

Near a low-residual optimum, the Hessian is closely related to the Gauss–Newton matrix

$$H(\theta^\star) \approx 2J_e(\theta^\star)^\top W J_e(\theta^\star),$$

where J_e is the Jacobian of residuals with respect to the calibrated parameters. This approximation is useful for interpretation: large eigenvalues correspond to directions in parameter space where the implied-volatility surface changes strongly, while small eigenvalues indicate flatter directions where different parameter values may produce similar quotes.

In this thesis, curvature diagnostics are used for three purposes:

- to check near-stationarity through $\|\nabla J(\theta^\star)\|_2$;
- to detect anisotropy through the Hessian eigenvalue spread and condition number;

- to identify cross-parameter couplings through the off-diagonal Hessian entries.

These diagnostics should be read locally. A positive semidefinite Hessian around $\theta^\star$ supports the interpretation of a locally stable basin, but it does not prove that the global inverse problem is unique. Similarly, a high condition number means that the calibration surface contains stiff and flat directions at the same time; it explains why residuals can be small even when some parameters are harder to recover accurately.

Placeholder: Jacobian and Hessian diagnostics
Candidate existing plots: `figures/cann/jacobian_theta_star_no_reg.png` and `figures/cann/hessian_theta_star_no_reg.png`. Suggested final layout: two panels, with the Jacobian vector on the left and the Hessian heatmap on the right.

Fig. 4.8. Placeholder for local first- and second-order diagnostics at the calibrated solution.

Placeholder: objective geometry / identifiability plot
Suggested plot: two-dimensional objective slices around $\theta^\star$, for example $(\bar{v}, v_0)$, (γ, v_0), and (ρ, γ). Purpose: visually explain stiff directions, flat valleys, and parameter coupling.

Fig. 4.9. Placeholder for objective-surface slices around the calibrated optimum.

The key implication is that calibration quality must be interpreted at two levels. The residual map measures how well the observed surface is reproduced. The Hessian and Jacobian diagnostics measure how informative that surface is about the individual model parameters. This distinction is crucial for the rest of the thesis: the downstream PINN only needs a market-consistent parameter vector, but parameter-level conclusions require additional identifiability evidence.

4.9. Connection with the Downstream PINN Stage

The calibrated vector $\theta^\star$ is the output of the CaNN stage. It is passed forward as the market state for later experiments:

$$\theta^\star = (\rho^\star, \kappa^\star, \gamma^\star, \bar{v}^\star, v_0^\star).$$

In the PINN formulation of Chapter 5, $(\rho^\star, \kappa^\star, \gamma^\star, \bar{v}^\star)$ define the Heston PDE operator, while $v_0^\star$ is used as the current variance state at inference time. This separation matters because the PINN is trained over a variance-state coordinate v, whereas calibration estimates the current initial variance v_0.

Consequently, the CaNN stage is not an isolated side experiment. It supplies the parameter bridge between market-implied-volatility fitting and model-consistent price/-Greek evaluation. The reliability of the downstream analysis therefore depends on both components: the ANN surrogate must be accurate in the calibration domain, and the calibration objective must produce a stable, interpretable $\theta^\star$.

Placeholder: CaNN to PINN interface
CaNN output $\theta^\star$ split into PDE parameters $(\rho^\star, \kappa^\star, \gamma^\star, \bar{v}^\star)$ and variance state $v_0^\star$;
then fed into the PINN input convention $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$.
This figure should make explicit that v is a PINN state variable, while $v_0^\star$ is fixed only at market evaluation.

Fig. 4.10. Placeholder for the interface between calibration and the PINN pricing/Greeks stage.

5. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL

This chapter reformulates vanilla option pricing under the Heston model as a PDE-constrained learning problem. In contrast to the supervised ANN surrogate of Chapter 3, the objective is no longer to emulate precomputed numerical labels, but to approximate directly the option-price function through an unsupervised Physics-Informed Neural Network (PINN). The methodology follows the general PINN philosophy and is inspired by the Heston-specific formulation proposed in [6], while adapting the state representation and notation to the conventions used in the present work.

5.1. Problem formulation

In this chapter, European option pricing under Heston is posed as the approximation of the solution of the pricing PDE rather than as a supervised regression problem. The neural network outputs the option price itself, and the training objective is to enforce that this output satisfies the Heston valuation equation together with its terminal and boundary conditions. Consequently, the core information driving the optimization is not a set of external labels, but the analytical structure of the model under the risk-neutral measure.

To avoid a notational clash between the price function and the variance state, the option value will be denoted by u, while the instantaneous variance state will be denoted by v. This distinction is important because the Heston model contains both a variance process and a pricing function, and using the same symbol for both would make the PDE formulation unnecessarily confusing. The unknown function approximated by the PINN is therefore written as

$$u_\phi(\tau, m, v, \Theta, r),$$

where ϕ denotes the trainable weights of the network, $m = S/K$ is moneyness, $\tau = T - t$ is time-to-maturity, v is the current instantaneous variance, and Θ collects the structural Heston parameters.

More precisely, the calibrated tuple obtained in the previous chapter is

$$\Theta^* = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

However, the role of v_0 must be distinguished from that of the other parameters. The quantities ρ, κ, γ and $\bar{v}$ define the operator of the Heston PDE, whereas v_0 is the initial condition of the variance process. In the PDE formulation, the state variable is the generic variance level v, not the fixed initial value v_0. Therefore, the parametric PINN is constructed to learn a family of Heston pricing solutions indexed by the structural parameters $(\rho, \kappa, \gamma, \bar{v}, r)$, while the current variance v_0 is used only when evaluating the trained network at the present market state. The calibrated market valuation is thus recovered through

$$u_\phi(\tau, m, v_0, \rho^*, \kappa^*, \gamma^*, \bar{v}^*, r).$$

This point is conceptually important: during training, the PINN learns the solution over a domain in $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, whereas at inference time the current market valuation is recovered by fixing the variance coordinate to the calibrated value v_0^* and the structural parameters to the calibrated tuple Θ^*.

The network is trained on collocation points sampled in the interior and on the relevant boundaries of the pricing domain. These points are not generated by simulating full paths of the Heston variance process. Instead, they are drawn directly from a bounded domain of admissible states, for instance using Latin Hypercube Sampling (LHS), and used to evaluate the PDE residual. In that sense, the method is unsupervised: the learning signal comes from the Heston structure itself, not from externally computed price labels. This is the main methodological difference with respect to the ANN surrogate developed in Chapter 3.

At a high level, the proposed workflow is the following. First, a parametric PINN is defined to approximate the price surface over a range of maturities, moneyness levels, variance states, and Heston parameters. Second, the PINN is trained by minimizing the residual of the Heston pricing PDE together with terminal and boundary mismatches. Third, once the network is trained, the price of a contract for a calibrated market configuration is obtained by evaluating the network at the corresponding $(\tau, m, v_0^*, \Theta^*, r)$. This preserves the link with the calibration pipeline while replacing label-based supervision with model-consistent unsupervised training. In this sense, the CaNN stage remains essential: it provides the market-specific parameter tuple that is consumed by the parametric PINN at inference time, without requiring a retraining of the pricing network for each new calibration.

- hablar muy bien de los inputs (heston params optimizados y tal)
- hablar sobre el output, que es V para vanilla option
- hablar sobre greeks por autodiff ?
- hablar de nuevo un poco sobre el pipeline

5.1.1. PINN Theoretical Framework

no estoy seguro de donde poner esto, o si quitarlo incluso

A PINN can be viewed as a parametric approximation of the solution of a differential equation. In the present setting, the network u_ϕ is required to satisfy the Heston pricing PDE in the interior of the domain and to match the contractual conditions at expiry and on selected boundaries. If $\mathcal{N}$ denotes the Heston differential operator, the target problem can be summarized abstractly as

$$\mathcal{N}[u] = 0 \quad \text{in the interior domain,} \qquad \mathcal{B}[u] = 0 \quad \text{on the boundaries,}$$

where $\mathcal{B}$ denotes the set of terminal and boundary constraints.

The PINN replaces the unknown analytical solution by a neural approximation u_ϕ and defines a residual function

$$\mathcal{R}_\phi = \mathcal{N}[u_\phi].$$

Training then consists of minimizing a composite loss built from three sources of error: the interior PDE residual, the mismatch with the terminal payoff, and the mismatch with the selected boundary conditions. In a compact notation, the loss takes the form

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi).$$

This decomposition is standard in the PINN literature and matches the structure adopted in [6].

An essential ingredient of this approach is automatic differentiation. Since the Heston PDE contains first- and second-order derivatives with respect to the state variables, the network output must remain differentiable with respect to its inputs. The required quantities are obtained by differentiating the computational graph of the network rather than by using finite-difference approximations. This is particularly convenient in the present case because the same differentiable surrogate can later be reused for the computation of Greeks.

- función aproximadora
- idea de residuo de la PDE
- concepto de la loss en una PINN

5.2. Heston Pricing PDE

Starting from the Heston dynamics introduced in Chapter 2 (Equation 2.11), the risk-neutral version used for pricing is

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(1)}, \tag{5.1}$$

$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(2)}, \tag{5.2}$$

with instantaneous correlation

$$d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\,dt.$$

Here r is the risk-free rate, κ is the mean-reversion speed of the variance process, $\bar{v}$ is the long-run variance level, γ is the volatility of variance, and ρ controls the correlation between returns and variance shocks. The current variance level at valuation time is denoted by v_0, so that v_t evolves from the initial condition $v(0) = v_0$.

Let $u(t, S, v)$ denote the price of a European vanilla option. By the Feynman-Kac representation, u satisfies the Heston pricing PDE [2]

$$u_t + \frac{1}{2}vS^2 u_{SS} + \rho\gamma v S u_{Sv} + \frac{1}{2}\gamma^2 v u_{vv} + rS u_S + \kappa(\bar{v} - v)u_v - ru = 0. \tag{5.3}$$

This is the continuous-time constraint that will be enforced by the PINN at interior collocation points. In the present chapter, the price function is denoted by u precisely to keep it separate from the variance state v.

For implementation purposes, it is more convenient to work with time-to-maturity $\tau = T - t$ and moneyness $m = S/K$ instead of calendar time t and spot S. Since the synthetic and market pipelines of this project use a fixed strike normalization $K = 1$, the pricing problem can be rewritten in terms of the state variables (τ, m, v). Defining the same pricing function in transformed coordinates, the PDE becomes

$$u_\tau = \frac{1}{2}vm^2 u_{mm} + \rho\gamma v m u_{mv} + \frac{1}{2}\gamma^2 v u_{vv} + rm u_m + \kappa(\bar{v} - v)u_v - ru. \tag{5.4}$$

This is the formulation that will be used to define the PDE residual in the training loss. It remains fully equivalent to the original Heston pricing equation, but it is better aligned with the representation used in the rest of the thesis.

For a European put option with normalized strike $K = 1$, the terminal condition at expiry is

$$u(0, m, v) = (1 - m)^+. \tag{5.5}$$

The payoff does not depend explicitly on the variance state, which means that the terminal condition is imposed uniformly over the v-dimension of the domain. In addition, a natural lower boundary condition is obtained at zero underlying value:

$$u(\tau, 0, v) = e^{-r\tau}. \tag{5.6}$$

For a general strike K, this condition would read $u(\tau, 0, v) = Ke^{-r\tau}$. In a first PINN specification, these two constraints are sufficient to reproduce the structure used in [6]: one term of the loss is attached to the interior PDE residual, one to the terminal payoff, and one to the lower boundary.

The far-field behavior as $m \to \infty$ for a put, namely $u(\tau, m, v) \to 0$, can also be incorporated if needed. However, to keep the initial implementation simple, the first version of the model will explicitly enforce only the terminal condition and the lower boundary, while controlling the remaining behavior through the bounded sampling domain in (τ, m, v) and the interior residual. This choice is consistent with the objective of building a minimal but coherent unsupervised PINN before adding more elaborate constraints.

- presentar la PDE de heston y hablar sobre ella

- definir la terminal condition payoff (depende de put/call !)
- hablar de las boundary conditions y cuales imponemos

5.3. Neural Network Architecture

5.3.1. Model Class

The pricing PINN is implemented as a fully connected feed-forward neural network (MLP) that approximates the mapping

$$u_\phi : \mathbb{R}^8 \to \mathbb{R}, \qquad x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r) \mapsto u_\phi(x),$$

where the scalar output is the option price. This architecture is appropriate for the present setting because all inputs are continuous tabular variables and no spatial or sequential inductive bias is required. citar referencia general sobre MLPs para PINNs

Compared with Chapter 3, where the ANN surrogate learned implied volatility from supervised labels, the role of the network here is to represent a differentiable pricing function that is constrained by the Heston PDE and boundary conditions. The model class is therefore selected not only for approximation capacity, but also for stable first- and second-order automatic differentiation. citar referencia sobre el papel de la diferenciabilidad en PINNs

5.3.2. Architecture Details

The reference architecture used in the implementation is

$$8 \to 256 \to 256 \to 256 \to 256 \to 1,$$

with `tanh` activation in hidden layers and linear activation in the output layer. In compact notation,

$$h^{(1)} = \varphi(W^{(1)}x + b^{(1)}), \ \ldots, \ h^{(4)} = \varphi(W^{(4)}h^{(3)} + b^{(4)}), \quad u_\phi = W^{(5)}h^{(4)} + b^{(5)},$$

with $\varphi = \tanh$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled and no batch-normalization layers are included. FIGURE: diagrama de arquitectura PINN (8-256-256-256-256-1, activaciones `tanh`, salida lineal)

TABLE 5.1. PINN ARCHITECTURE USED IN THIS CHAPTER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	256 neurons per layer
Hidden activation	`tanh`
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Own implementation (`configs/pinn_model_architecture.yaml`)

5.3.3. Model Inputs and Outputs

The model receives the following ordered input vector:

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r),$$

where (τ, m, v) are state variables, r is the risk-free rate, and $(\rho, \kappa, \gamma, \bar{v})$ are the structural Heston parameters that define the PDE operator. TABLE: resumen de inputs del PINN (variable, significado financiero, rol en PDE, rango de entrenamiento)

The network output is a scalar price $u_\phi(x)$, corresponding to the normalized-strike vanilla contract considered in this chapter ($K = 1$). This output choice is consistent with the PDE-constrained formulation: the PINN directly approximates the pricing function rather than an implied-volatility transformation. FIGURE: esquema de entradas/salida y conexión CaNN → PINN en inferencia

In the current experimental setup, collocation generation is run in a fixed-parameter regime for $(\rho, \kappa, \gamma, \bar{v})$, while (τ, m, v, r) vary across points. This means the model remains formally 8-dimensional, but some coordinates may be constant in a given run due to calibration-conditioned sampling. aclarar si este apartado se mantiene como baseline fixed-theta o si se añade variante fully parametric

Input Scaling

Although the pricing problem is formulated in physical variables, the optimization is performed with an affine rescaling of the network inputs. The motivation is numerical: (τ, m, v) and the Heston-related coordinates do not share comparable magnitudes, which can deteriorate conditioning and gradient balance. Following practical PINN recommendations, centering and scaling are used as a stabilization device. citar referencia sobre scaling en PINNs

In the implemented pipeline, the transformation is applied componentwise to the full PINN input vector through

$$\tilde{x}_d = a_d + b_d x_d, \qquad d = 1, \ldots, 8.$$

The coefficients are estimated from the collocation points of the training split:

$$\mu_d = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_{i,d}, \qquad \sigma_d = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_{i,d} - \mu_d)^2},$$

and then

$$a_d = -\frac{\mu_d}{\sigma_d}, \qquad b_d = \frac{1}{\sigma_d}.$$

To avoid numerical issues on nearly constant coordinates, the implementation uses a small threshold $\varepsilon = 10^{-8}$: if $\sigma_d \leq \varepsilon$, that coordinate is left unscaled ($a_d = 0,\ b_d = 1$).

The PDE residual is still defined in physical variables and differentiated by automatic differentiation. In code, the network is evaluated at $\tilde{x}$, while derivatives are taken with respect to the original coordinates x; the chain rule is therefore handled by the computational graph. Consequently, input scaling changes the numerical conditioning of training, but not the financial meaning of the PDE constraint. FIGURE: comparación visual antes/después del escalado en los inputs principales

5.4. Training Procedure

5.4.1. Loss Function

The PINN is trained by minimizing a composite loss that enforces the pricing PDE in the interior of the domain together with the terminal payoff and the lower boundary condition. Let $u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ denote the output of the neural network. From the transformed Heston PDE introduced above, the residual at an interior point is defined as

$$\mathcal{R}_\phi = u_\tau - \frac{1}{2} v m^2 u_{mm} - \rho \gamma v m u_{mv} - \frac{1}{2} \gamma^2 v u_{vv} - r m u_m - \kappa(\bar{v} - v) u_v + r u, \tag{5.7}$$

where all derivatives are evaluated by automatic differentiation on the computational graph of the network. In the ideal case, the exact pricing function would satisfy $\mathcal{R}_\phi = 0$ for every admissible interior point.

Let $\mathcal{S}_D = \{z_j\}_{j=1}^{N_D}$ denote the set of interior collocation points, with

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j).$$

The interior loss is the mean squared residual

$$\mathcal{L}_{\text{PDE}}(\phi) = \frac{1}{N_D} \sum_{j=1}^{N_D} \mathcal{R}_\phi(z_j)^2. \tag{5.8}$$

On the terminal set $\mathcal{S}_T = \{z_k^T\}_{k=1}^{N_T}$, where $\tau = 0$, the network must match the payoff of the European put. The corresponding error is

$$e_k^T(\phi) = u_\phi(0, m_k, v_k, \rho_k, \kappa_k, \gamma_k, \bar{v}_k, r_k) - (1 - m_k)^+, \tag{5.9}$$

and the terminal loss is defined by

$$\mathcal{L}_{\text{term}}(\phi) = \frac{1}{N_T} \sum_{k=1}^{N_T} \left(e_k^T(\phi) \right)^2. \tag{5.10}$$

On the lower boundary set $\mathcal{S}_{\text{low}} = \{z_l^{\text{low}}\}_{l=1}^{N_{\text{low}}}$, where $m = 0$, the exact put price is the discounted strike. Since the implementation uses the normalized strike $K = 1$, the lower-boundary error is

$$e_l^{\text{low}}(\phi) = u_\phi(\tau_l, 0, v_l, \rho_l, \kappa_l, \gamma_l, \bar{v}_l, r_l) - e^{-r_l \tau_l}, \tag{5.11}$$

and the associated loss is

$$\mathcal{L}_{\text{bc}}(\phi) = \frac{1}{N_{\text{low}}} \sum_{l=1}^{N_{\text{low}}} \left(e_l^{\text{low}}(\phi) \right)^2. \tag{5.12}$$

The total training objective is then written as

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi). \tag{5.13}$$

For implementation, it is convenient to keep an explicit weighted form,

$$\mathcal{L}_{\text{PINN}}(\phi) = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\phi) + \lambda_{\text{term}} \mathcal{L}_{\text{term}}(\phi) + \lambda_{\text{low}} \mathcal{L}_{\text{bc}}(\phi), \tag{5.14}$$

with non-negative coefficients $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$ that can be tuned if one component dominates the optimization. In the baseline specification of this thesis, all three weights are initialized to one, i.e. $(1, 1, 1)$, following the simplest formulation in [6].

A practical point is that the PDE term requires second-order derivatives with respect to inputs. Therefore, the automatic-differentiation graph must be kept when computing first derivatives, so that mixed and second derivatives such as u_{mv} and u_{vv} can be obtained consistently in the same computational graph. This is the key implementation detail that enables direct optimization of the PDE residual without finite-difference approximations.

5.4.2. Optimizer and Hyperparameters

The optimization follows a two-stage strategy that is common in PINN practice: an initial stochastic phase with Adam, followed by a quasi-Newton refinement with L-BFGS. In this work, this strategy is implemented as a mixed schedule in which Adam is used during the first part of training and L-BFGS in the second part, while a pure Adam or pure L-BFGS run remains available as an ablation.

Since the three loss terms are evaluated on different sample sets, each epoch uses three mini-batches: one from $\mathcal{S}_D$ (interior), one from $\mathcal{S}_T$ (terminal), and one from $\mathcal{S}_{\text{low}}$ (lower boundary). This allows different batch sizes for interior and boundary constraints and keeps the training loop aligned with the composite objective.

The reference hyperparameter block for the first implementation includes: learning rates and optimizer-specific settings, the Adam-to-L-BFGS switching epoch in mixed mode, batch sizes for interior and boundary sets, and the triplet of loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$.

For diagnostics, the training run stores epoch-wise train/validation curves for the total weighted loss and for each component separately. In addition, a two-dimensional error map over (m, τ) is produced from validation interior points by averaging $|\mathcal{R}_\phi|$ in bins. This map helps identify whether specific regions of the contract domain remain systematically underfitted even when the global loss decreases.

5.4.3. Sampling strategy

Training requires three different sample sets, each associated with one component of the loss. As in the general PINN methodology, these sets are not built from labeled prices but from points drawn directly in the state-parameter domain. The first set corresponds to the interior of the PDE domain, the second to the terminal boundary at expiry, and the third to the lower boundary at zero underlying value.

The interior domain is defined over bounded intervals for the state variables and for the Heston parameters:

$$\tau \in [0, \tau_{\max}], \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

combined with admissible ranges for $(\rho, \kappa, \gamma, \bar{v}, r)$. These parameter intervals should remain consistent with the calibration regime studied in the previous chapters, since the purpose of the parametric PINN is precisely to generalize over the family of Heston configurations that may be produced by the CaNN stage. In particular, the v-dimension is sampled explicitly as a state coordinate of the PDE. This is a crucial point: the PINN is trained on generic variance states v, whereas the calibrated initial variance v_0 is only used later, when evaluating the trained network at the current market state.

To obtain a representative coverage of this multidimensional domain with a controlled number of points, the baseline sampling method will be Latin Hypercube Sampling (LHS). Compared with naive Monte Carlo sampling, LHS provides a more regular coverage of each marginal dimension and reduces the risk of leaving large regions of the domain underexplored. In the present setting, this is particularly useful because the PINN must generalize simultaneously over state variables and model parameters.

For the interior set $\mathcal{S}_D$, each sampled point has the form

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j), \qquad j = 1, \dots, N_D.$$

These are the collocation points at which the PDE residual is evaluated. For the terminal set $\mathcal{S}_T$, points are sampled in the subdomain

$$\tau = 0, \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

again jointly with the Heston parameters. This set is used to impose the payoff condition. For the lower-boundary set $\mathcal{S}_{\text{low}}$, the sampled points satisfy

$$m = 0, \qquad \tau \in [0, \tau_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

with the same parameter ranges, and are used to enforce the discounted-strike boundary condition.

From an implementation viewpoint, the three sets can be regenerated dynamically during training or precomputed once and reused across epochs. A dynamic resampling strategy may improve coverage of the domain and reduce overfitting to a fixed cloud of collocation points, whereas a static design simplifies reproducibility and debugging. The initial implementation will prioritize simplicity and reproducibility; therefore, the first reference version will rely on fixed sample sets generated before the training loop. Dynamic resampling can later be evaluated as an extension if it leads to more stable convergence.

This sampling strategy preserves the fully unsupervised nature of the method. No prices from COS, FFT, or Monte Carlo are required to build the training objective. Numerical pricing methods remain useful for ex-post validation and benchmarking, but not as a source of labels for fitting the PINN itself.

5.5. Extension: Sobolev-Enhanced Training for Greek Accuracy

The baseline PINN objective in this chapter is designed to recover prices by enforcing the Heston PDE and boundary/terminal conditions. However, accurate prices do not automatically imply accurate first- and second-order sensitivities. In particular, Γ and variance/rate sensitivities may show larger local errors because they depend on curvature properties of the learned surface. To address this limitation, a natural extension is to enrich training with derivative-aware terms inspired by Sobolev training [18] and gradient-enhanced PINN formulations [7]. The formulation below is an application-specific hybrid: it follows Sobolev Training in matching target derivatives, but unlike gPINN it does not penalize gradients of the PDE residual directly.

The extension keeps the original physics objective and adds supervised derivative terms on a dedicated set of Greek-anchor points. Let

$$\mathcal{L}_{\text{base}} = \lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{term}}\mathcal{L}_{\text{term}} + \lambda_{\text{low}}\mathcal{L}_{\text{low}},$$

and let $\mathcal{S}_{\text{g}}$ be a set of anchor points in the same input space

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

For each $x \in \mathcal{S}_\mathrm{g}$, reference sensitivities $\mathcal{G}^\star(x)$ are obtained from the semi-analytical Heston characteristic-function map (Section 2.6.2), while $\mathcal{G}_\phi(x)$ are computed from autodiff on the PINN output.

To prevent scale dominance across Greeks, we use normalized residuals and split supervision into two blocks:

$$\mathcal{L}_{\mathrm{g},1} = \frac{1}{|\mathcal{S}_\mathrm{g}|\,|Q_1|} \sum_{x\in\mathcal{S}_\mathrm{g}} \sum_{q\in Q_1} \left(\frac{q_\phi(x) - q^\star(x)}{s_q + \varepsilon} \right)^2, \qquad Q_1 = \{\Delta, \mathrm{Vega}_\nu, \Theta, \mathrm{Rho}\}, \tag{5.15}$$

$$\mathcal{L}_{\mathrm{g},2} = \frac{1}{|\mathcal{S}_\mathrm{g}|} \sum_{x\in\mathcal{S}_\mathrm{g}} \left(\frac{\Gamma_\phi(x) - \Gamma^\star(x)}{s_\Gamma + \varepsilon} \right)^2, \tag{5.16}$$

where s_q are fixed scale factors (e.g., robust quantiles of $|q^\star|$ on $\mathcal{S}_\mathrm{g}$) and $\varepsilon > 0$ avoids numerical singularities near zero. The separation of $\mathcal{L}_{\mathrm{g},2}$ for Γ gives independent control of curvature supervision, which is typically more unstable than first-order terms.

The prototype objective is then

$$\mathcal{L}_{\mathrm{proto}} = \mathcal{L}_{\mathrm{base}} + \lambda_{\mathrm{g},1}\mathcal{L}_{\mathrm{g},1} + \lambda_{\mathrm{g},2}\mathcal{L}_{\mathrm{g},2}. \tag{5.17}$$

From an optimization viewpoint, this extension introduces additional multi-term imbalance risks, already documented in the PINN literature [8], [9], [10]. Therefore, the intended training strategy is two-stage: first, train the baseline PINN to a stable price solution; second, apply a short fine-tuning phase with gradually increased $(\lambda_{\mathrm{g},1}, \lambda_{\mathrm{g},2})$, while monitoring possible regressions in price/PDE metrics.

This section defines a methodological prototype for subsequent work. It is included to fix the exact formulation to be tested, but it is not yet part of the benchmarked implementation reported in this thesis.

5.6. Greeks Computation

5.6.1. Definition of Greeks from the PINN Output

Once the PINN has been trained, the network output is a differentiable approximation of the vanilla option price:

$$u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

Therefore, sensitivities can be obtained directly as partial derivatives of this function, evaluated at a chosen market point. Let

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

The first-order and second-order derivatives with respect to the input coordinates are denoted by

$$\nabla_x u_\phi(x), \qquad \nabla_x^2 u_\phi(x).$$

From these objects, standard trading Greeks are extracted as follows:

$$\Delta = \frac{\partial u_\phi}{\partial S}, \tag{5.18}$$

$$\Gamma = \frac{\partial^2 u_\phi}{\partial S^2}, \tag{5.19}$$

$$\Theta = \frac{\partial u_\phi}{\partial t} = -\frac{\partial u_\phi}{\partial \tau}, \tag{5.20}$$

$$\text{Rho} = \frac{\partial u_\phi}{\partial r}. \tag{5.21}$$

In this work, the spatial coordinate is moneyness $m = S/K$, not spot directly. Hence a chain-rule conversion is required:

$$\frac{\partial u_\phi}{\partial S} = \frac{\partial u_\phi}{\partial m}\frac{\partial m}{\partial S} = \frac{1}{K}\frac{\partial u_\phi}{\partial m}, \tag{5.22}$$

$$\frac{\partial^2 u_\phi}{\partial S^2} = \frac{1}{K^2}\frac{\partial^2 u_\phi}{\partial m^2}. \tag{5.23}$$

For the normalized-strike setup used in the thesis ($K = 1$), both coordinate systems coincide numerically.

Since the PINN state includes instantaneous variance v, a natural model-consistent volatility sensitivity is

$$\text{Vega}_v := \frac{\partial u_\phi}{\partial v}.$$

If needed, this can be transformed to a sensitivity with respect to volatility level $\sigma_v = \sqrt{v}$:

$$\frac{\partial u_\phi}{\partial \sigma_v} = 2\sqrt{v}\,\frac{\partial u_\phi}{\partial v}.$$

In addition, the parametric PINN provides direct structural sensitivities

$$\frac{\partial u_\phi}{\partial \rho}, \quad \frac{\partial u_\phi}{\partial \kappa}, \quad \frac{\partial u_\phi}{\partial \gamma}, \quad \frac{\partial u_\phi}{\partial \bar{v}},$$

which are useful for risk decomposition and calibration diagnostics beyond classical desk Greeks.

5.6.2. Automatic Differentiation of the PINN

The derivative computation is based on automatic differentiation over the computational graph of the trained network. For a single input point x, we compute:

$$u_\phi(x), \qquad J(x) = \nabla_x u_\phi(x), \qquad H(x) = \nabla_x^2 u_\phi(x).$$

Because the model is differentiable end-to-end, these quantities are exact derivatives of the neural surrogate (up to floating-point precision), not finite-difference approximations.

Compared with bump-and-reprice finite differences, this approach has three practical advantages in the PINN context:

- it avoids step-size tuning and the associated truncation/cancellation trade-off;
- it provides first- and second-order sensitivities in a single coherent graph;
- it scales efficiently to batched inputs through vectorized Jacobian/Hessian evaluation.

From a financial perspective, the interpretation remains standard: each Greek is the local slope or curvature of the price surface produced by the PINN around the evaluation point. The quality of those sensitivities therefore depends on both the local smoothness of the trained network and the fidelity of the learned price function in that region. This closes the link with the Heston dynamics introduced in Chapter 2: the PINN price surface is trained through the PDE implied by Equation 2.11, and the Greeks are then derivatives of that model-consistent learned surface.

6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

6.1. Experimental Setup And Evaluation Protocol

This chapter compares the three model blocks developed in the thesis under a common evaluation protocol: ANN pricing surrogate, calibration network workflow, and PINN-based pricing and Greeks computation.

Unless explicitly stated, all reported metrics are computed on fixed evaluation sets with identical feature conventions, consistent preprocessing, and the same numerical references used in previous chapters.

The minimal pre-benchmark validation protocol combines three checks: convergence with respect to (N, L) in the COS expansion, put-call parity residual control across strikes, and Black–Scholes consistency in near-constant-variance regimes.

These checks are used as a guardrail before comparing models, so that downstream differences can be attributed to model behavior rather than to numerical artifacts. For consistency across Sections 6.2–6.5, each block is analyzed with the same reading logic: global summary metrics, localized error diagnostics over relevant domains, and runtime indicators.

The protocol is fixed as follows. Randomness is controlled with a common seed (42) in data generation and training configurations; in calibration, the Differential Evolution seed follows the same base value unless explicitly overridden. The Heston domain is shared across experiments, with $\rho \in [-0.9, 0]$, $\kappa \in [0, 3]$, $\gamma \in [0.01, 0.8]$, $\bar{v} \in [0.01, 0.5]$, $v_0 \in [0.05, 0.5]$, and contract coordinates $m \in [0.6, 1.4]$, $\tau \in [0.05, 3.0]$ for ANN/CaNN benchmarks. Reference numerics are also fixed: COS pricing with $N = 1500$, $L = 50$, and fallback $L = 80$ when required by stability checks, plus Brent IV inversion with $\text{tol} = 10^{-8}$ and bounded search interval.

Reported metrics are always computed on fixed held-out sets, with ANN split policy fixed to $0.8/0.1/0.1$ (train/validation/test), and values reported in decimal units (IV for ANN/CaNN, price for PINN). Finally, each model block reports equivalent evidence layers (global table, regional/spatial diagnostic, and convergence or runtime indicator), so cross-model comparisons remain methodologically aligned.

Methodological details of each block are kept in their dedicated chapters (Chapter 3 for ANN, Chapter 4 for CaNN, and Chapter 5 for PINN), while this chapter focuses on consolidated benchmark outcomes under the protocol above.

6.2. ANN Pricer Performance

This section reports the performance of the supervised ANN pricer introduced in Chapter 3. The objective is to assess mean implied-volatility accuracy and spatial stability over (m, τ), using a fully specified evaluation protocol that can be reproduced without implementation-specific references.

The ANN analyzed here corresponds to the selected baseline configuration from Chapter 3. The benchmark uses fixed weights (no retraining in Chapter 6) and fixed train/validation/test splits of sizes 800,000/100,000/100,000. Pointwise error is defined as

$$e = \hat{\sigma}_{\mathrm{IV}} - \sigma_{\mathrm{IV}}^{\mathrm{ref}},$$

where $\hat{\sigma}_{\mathrm{IV}}$ is the ANN-predicted implied volatility and $\sigma_{\mathrm{IV}}^{\mathrm{ref}}$ is the reference implied volatility obtained from the Heston-COS/Brent pipeline. The error e is reported in decimal implied-volatility units.

Global quality is summarized with RMSE, MAE, and $p_{90}(|e|)$. Regional diagnostics use fixed bins over m and τ, and extrapolation behavior is analyzed through sensitivity maps with explicit interpolation limits (Figure 6.1).

6.2.1. Global Error Metrics

Table 6.1 reports split-wise global errors for the selected ANN baseline.

TABLE 6.1. GLOBAL ANN IMPLIED-VOLATILITY ERRORS (BASELINE CONFIGURATION).

Split	**n**	**RMSE**	**MAE**	$\mathbf{p_{90}(\lvert e \rvert)}$
Train	800,000	5.44×10^{-4}	3.82×10^{-4}	8.15×10^{-4}
Validation	100,000	2.16×10^{-3}	4.11×10^{-4}	8.27×10^{-4}
Test	100,000	3.03×10^{-3}	4.29×10^{-4}	8.28×10^{-4}

Source: Own elaboration

The test errors remain small in absolute IV units, and the upper-quantile behavior (p_{90}) stays in the 10^{-3} range, which is consistent with a stable surrogate in the interpolation region.

Figure 6.1 provides a panel-wise diagnostic. Each row corresponds to one parameter plane: top $(\gamma, \bar{v})$ (vol-of-vol versus long-run variance), middle (ρ, γ) (correlation versus vol-of-vol), and bottom $(\kappa, \bar{v})$ (mean reversion versus long-run variance). The left column shows ANN IV maps with white isoclines, while the right column shows $\log|\mathrm{NN} - \mathrm{Heston}|$ on the same grids. The dashed black lines delimit interpolation limits; regions beyond those limits are extrapolation zones where the network was not trained.

The most stable behavior is observed in the interior of each plane, where isoclines are smooth and error colors remain comparatively mild. Error increases near boundary segments and especially outside the dashed limits, with the strongest deterioration concentrated in outer corners of the tested planes. This is consistent with the regional-error reading: the surrogate remains accurate over the interpolation core, while low-moneyness/short-maturity-like boundary regimes are the most fragile. In practical terms, Figure 6.1 supports using the ANN as a robust interpolation surrogate and treating extrapolated regions as lower-confidence zones.

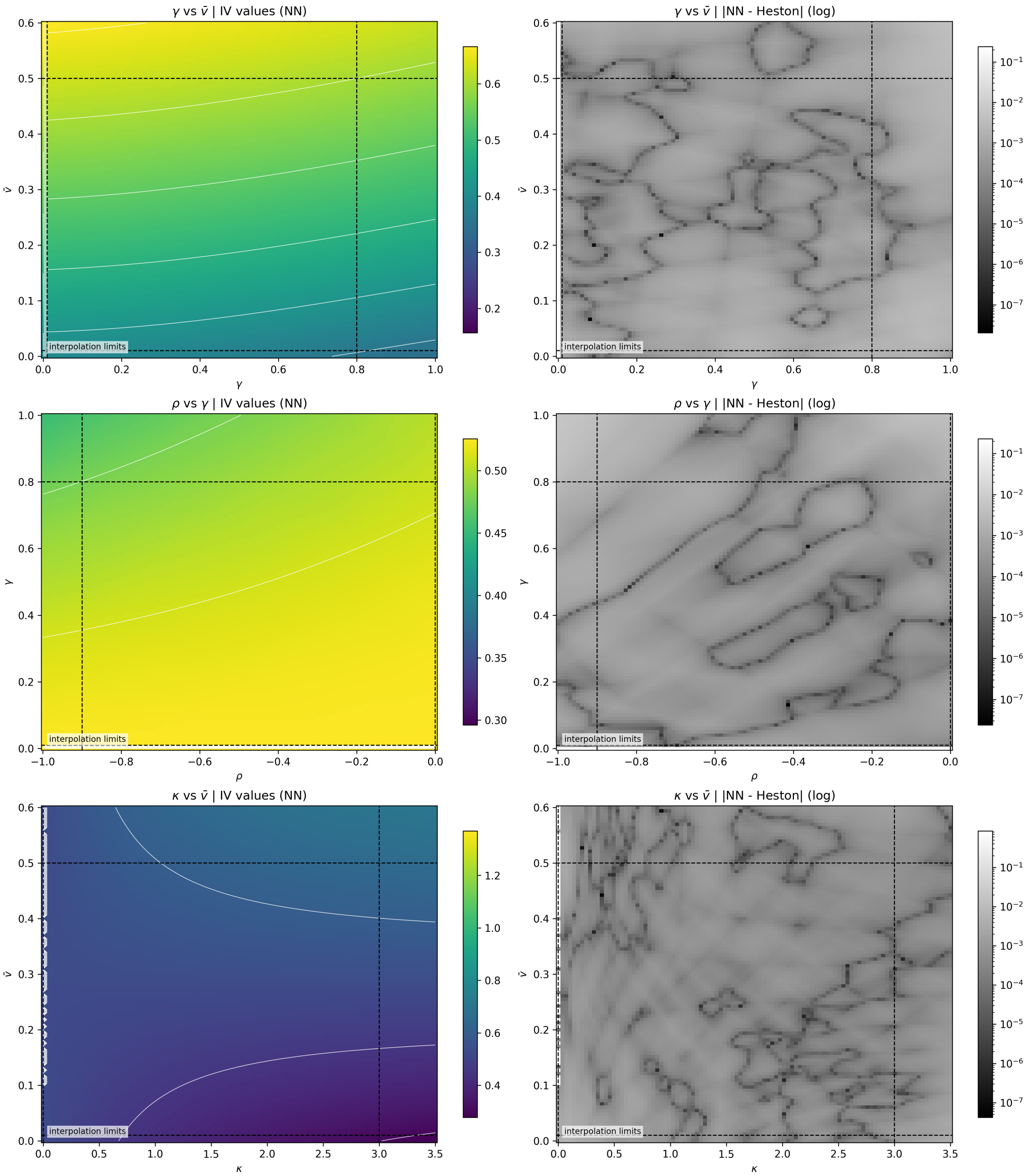

Fig. 6.1. Sensitivity grid for the ANN pricer. Top row: $(\gamma, \bar{v})$, middle row: (ρ, γ), bottom row: $(\kappa, \bar{v})$. Left panels report IV-value maps with white isoclines; right panels report $\log|\mathrm{NN} - \mathrm{Heston}|$. Dashed black lines indicate interpolation limits.

MEJORAR ESTA FIGURA

6.2.2. Regional Error Profile

Regional diagnostics identify where approximation error concentrates. Table 6.2 reports representative bins from the test split; *Share (%)* indicates the fraction of test points that fall into each bin, so error magnitude can be interpreted jointly with sample coverage.

TABLE 6.2. REPRESENTATIVE REGIONAL ANN ERRORS ON THE TEST SPLIT.

Feature	Bin	Share (%)	RMSE	MAE
m	$[0.6, 0.8]$	25.11	5.98×10^{-3}	6.11×10^{-4}
τ	$[0.05, 0.25]$	6.79	1.15×10^{-2}	1.23×10^{-3}
m	$[1.1, 1.2]$	12.30	4.73×10^{-4}	3.51×10^{-4}
τ	$[2.0, 3.0]$	33.89	4.45×10^{-4}	3.23×10^{-4}

Source: Own elaboration

The reading is the following. The high-error bins are concentrated at low moneyness and short maturity ($[0.6, 0.8]$ in m, $[0.05, 0.25]$ in τ). The two lower-error rows represent interior/longer-horizon regions and show a clear error drop. This pattern is therefore structural, not a rare-point artifact: the most difficult moneyness bin still carries a large share of the test set (25.11%).

Figure 6.2 makes the same pattern explicit with two panels: left panel (MSE by moneyness bins) and right panel (MSE by maturity bins). The figure reports only bin-level MSE curves (train/validation/test); it does not display sample coverage. Coverage is exactly what the *Share (%)* column in Table 6.2 represents. In both validation and test curves, the first bin is clearly the most difficult, while errors drop sharply and remain much lower in interior bins. This is consistent with boundary/extrapolation effects and with the smoother behavior observed around central bins.

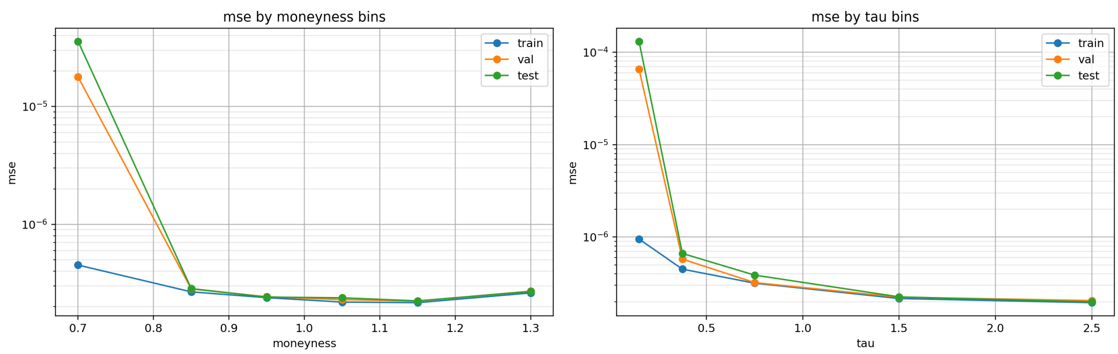

Fig. 6.2. Regional ANN MSE by moneyness bins (left) and maturity bins (right). Curves correspond to train, validation, and test.

MEJORAR ESTA FIGURA

6.3. Calibration Network Performance

This section reports the benchmark outcomes of the CaNN workflow. The full calibration methodology (problem formulation, optimizer mechanics, and detailed design choices) is developed in Chapter 4; here we focus on final metrics and on the selection of the reference calibration run used in the rest of the chapter.

6.3.1. Benchmark Snapshot

The calibration benchmark in this section uses a fixed ANN surrogate (from Section 6.2) and one fixed quote universe of 35 contracts (Table 6.3). The calibrated vector is $(\rho, \kappa, \gamma, \bar{v}, v_0)$, optimized with Differential Evolution plus polish under the same bounds and controls defined in Chapter 4.

TABLE 6.3. QUOTE UNIVERSE USED IN THE CANN BENCHMARK.

Item	Value
Number of quotes	35
Grid structure	Complete 5×7 grid over (m, τ)
Moneyness nodes m	$\{0.85, 0.925, 1.00, 1.075, 1.15\}$
Maturity nodes τ	$\{0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00\}$
Interest rate r	0.03 (constant across quotes)

Source: Own elaboration

6.3.2. Fit Quality And Convergence

A compact regularization mini-study was run with the same quote set and optimizer controls. The no-regularization variant ($\lambda = 0$) yields residual RMSE 2.01441×10^{-4}, while the L2 variant ($\lambda = 10^{-6}$) yields 2.01445×10^{-4}. The difference is marginal, but the no-regularization run is slightly better and is therefore selected as the reference configuration for the remainder of Section 6.3.

To characterize residual magnitudes directly in IV units, Table 6.4 summarizes the absolute-residual distribution over the 35 quotes for the selected run.

TABLE 6.4. ABSOLUTE-RESIDUAL DISTRIBUTION ACROSS MARKET QUOTES (SELECTED RUN).

Mean $\lvert r \rvert$	**Median** $\lvert r \rvert$	$\mathbf{p_{95}(\lvert r \rvert)}$	**Max** $\lvert r \rvert$
1.70×10^{-4}	1.58×10^{-4}	3.30×10^{-4}	4.52×10^{-4}

Source: Own elaboration

Residual concentration is mild. The largest local deviations appear around intermediate-to-high moneyness and medium maturities, but their scale remains below 5×10^{-4} in absolute IV units.

Figure 6.3 shows the signed residual map over (m, τ) for the reference no-regularization solution. The color pattern is balanced around zero (no strong systematic bias), with localized positive and negative pockets whose magnitude remains in the same 10^{-4} scale reported in Table 6.4.

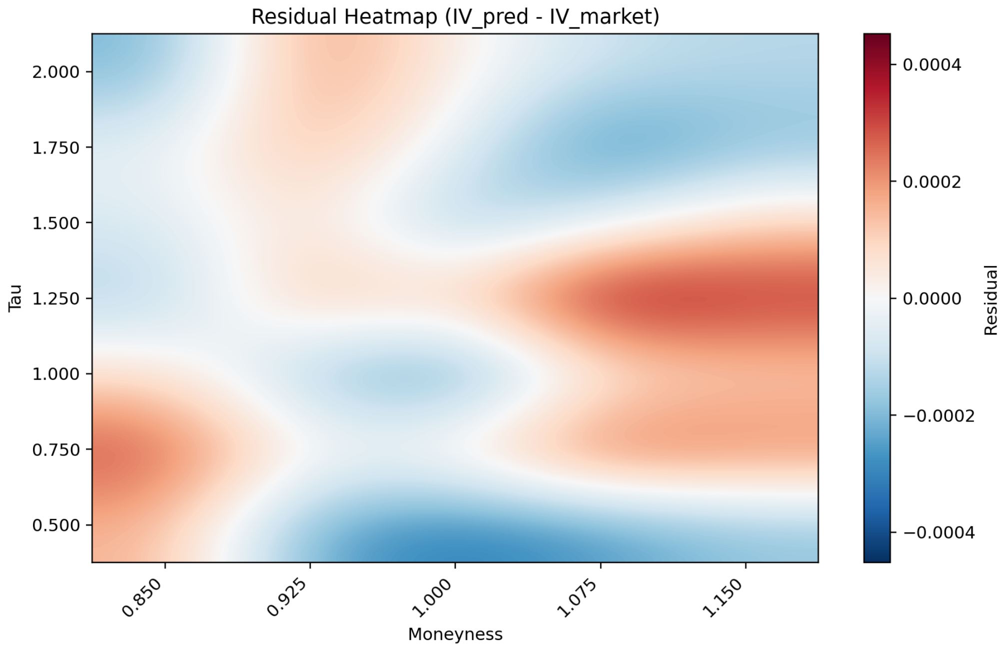

Fig. 6.3. Signed residual heatmap over (m, τ) for the CaNN reference calibration (no regularization). Residual defined as $\mathrm{IV}_{\mathrm{pred}} - \mathrm{IV}_{\mathrm{market}}$. Rendering is visually smoothed for readability; values come from the original quote grid.

MEJORAR ESTA FIGURA

6.3.3. Parameter Recovery Profile

Since this quote set includes sidecar synthetic truth, parameter-level errors can be reported for the reference no-regularization solution.

TABLE 6.5. PARAMETER ERROR PROFILE FOR THE REFERENCE CANN CALIBRATION (NO REGULARIZATION).

Parameter	True value	Abs. error	Rel. error
ρ	-0.45	7.41×10^{-2}	1.65×10^{-1}
κ	1.00	5.76×10^{-2}	5.76×10^{-2}
γ	0.35	5.04×10^{-2}	1.44×10^{-1}
$\bar{v}$	0.20	2.42×10^{-3}	1.21×10^{-2}
v_0	0.22	2.89×10^{-4}	1.31×10^{-3}

Source: Own elaboration

The largest relative deviations appear in ρ and γ, while $\bar{v}$ and v_0 are recovered with substantially smaller error.

TABLE 6.6. SELECTED CANN RUN VERSUS LIU ET AL. (ABSOLUTE PARAMETER ERRORS).

Metric	This work (selected run)	Liu et al. [5]
Weighted MSE	4.06×10^{-8}	$\sim 10^{-8}$
$\lvert\rho - \rho^\star\rvert$	7.41×10^{-2}	4.84×10^{-2}
$\lvert\kappa - \kappa^\star\rvert$	5.76×10^{-2}	4.88×10^{-2}
$\lvert\gamma - \gamma^\star\rvert$	5.04×10^{-2}	3.28×10^{-2}
$\lvert\bar{v} - \bar{v}^\star\rvert$	2.42×10^{-3}	4.54×10^{-3}
$\lvert v_0 - v_0^\star\rvert$	2.89×10^{-4}	4.39×10^{-4}

Source: Own elaboration based on selected run outputs and [5].

In Table 6.6, *Weighted MSE* denotes the weighted mean-squared IV residual used as calibration objective (same weighting convention as Chapter 4). The pattern is consistent across both studies: $(\bar{v}, v_0)$ are recovered more accurately than (ρ, γ). The comparison is informative but not a strict ranking, because quote set design, optimization budget, and implementation details are not identical.

6.3.4. Local Smoothness And Curvature Study

To characterize local identifiability around the calibrated optimum, Jacobian and Hessian diagnostics were computed at $\theta^\star$ for the selected run.

TABLE 6.7. LOCAL-CURVATURE DIAGNOSTICS AT THE CALIBRATED SOLUTION $\theta^\star$ (SELECTED RUN).

$\|\nabla J(\theta^\star)\|_2$	$\lambda_{\min}(H)$	$\lambda_{\max}(H)$	$\kappa_{\text{abs}}(H)$
6.87×10^{-6}	5.15×10^{-4}	4.16×10^{1}	8.07×10^{4}

Source: Own elaboration

The selected solution satisfies near-stationarity ($\|\nabla J(\theta^\star)\|_2 \ll 10^{-4}$). The Hessian is symmetric, and the reported extreme eigenvalues are both non-negative in this run, consistent with a locally convex basin around the selected optimum. At the same time, $\kappa_{\text{abs}}(H) \approx 8 \times 10^4$ indicates pronounced anisotropy (flat and stiff directions coexist). The strongest cross-parameter coupling appears between $\bar{v}$ and v_0 (off-diagonal entry ≈ 18.69), followed by (γ, v_0) and $(\gamma, \bar{v})$, which is coherent with the observed recovery pattern.

Figure 6.4 shows the Jacobian components at $\theta^\star$ for the no-regularization reference run. All components remain close to zero (between 10^{-8} and 10^{-6}), reinforcing first-order convergence. The largest entries are associated with $\bar{v}$ and v_0, but their magnitude is still very small, so there is no dominant residual slope left at the optimum.

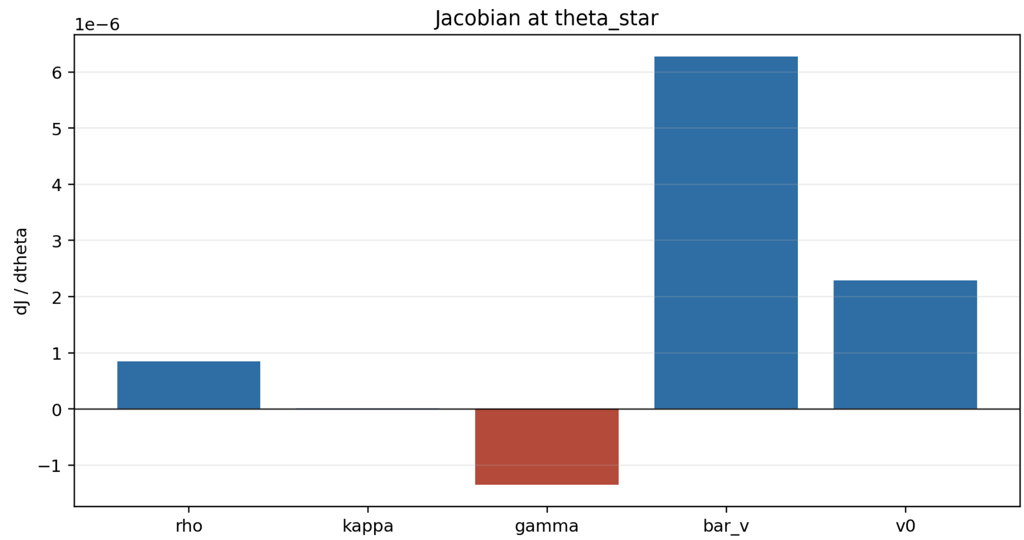

Fig. 6.4. Jacobian components at $\theta^\star$ for the CaNN reference calibration (no regularization).

MEJORAR ESTA FIGURA

Figure 6.5 is the key second-order diagnostic. The heatmap makes explicit a block structure centered on $(\bar{v}, v_0)$: both diagonal entries are the largest (high local curvature), and their off-diagonal term is also large, which indicates strong local coupling between long-run variance and initial variance. By contrast, the κ-related diagonal curvature is much smaller, so perturbations in that direction are less strongly penalized around $\theta^\star$. This mixed pattern (stiff variance block plus flatter directions in other coordinates) explains why fit residuals can be very small while some parameters, especially ρ and γ, remain harder to pin down with the same precision as $(\bar{v}, v_0)$. Comentario visual: conservar esta figura como diagnóstico principal de curvatura local en CaNN.

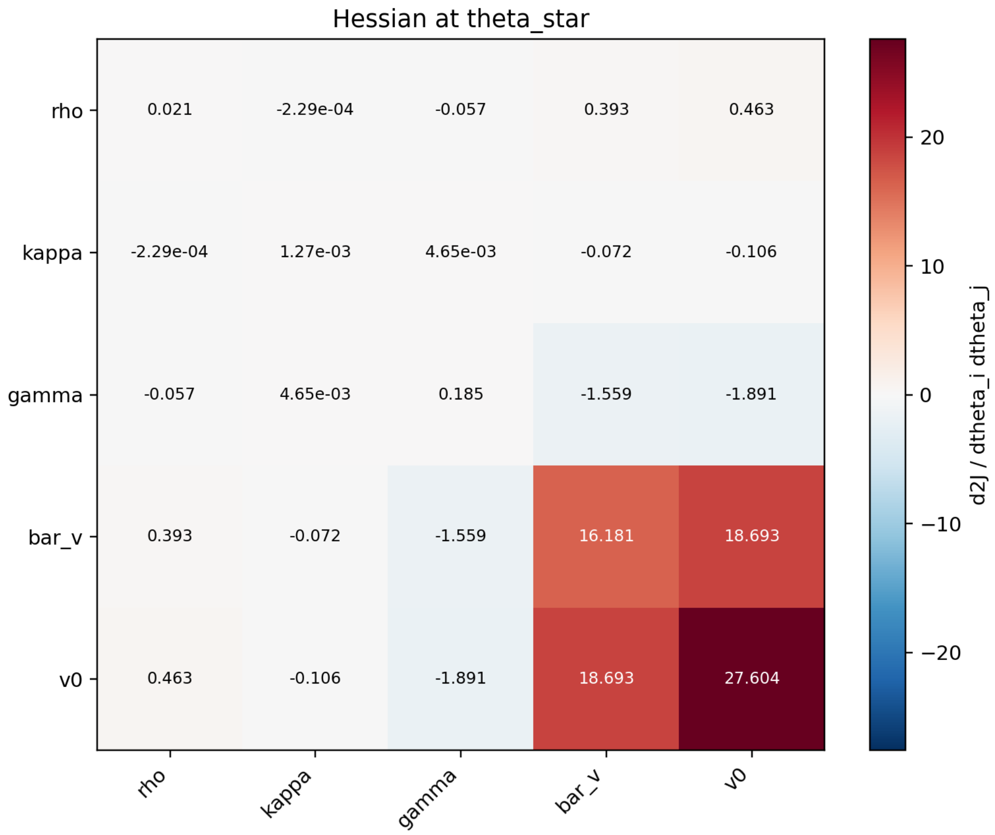

Fig. 6.5. Hessian matrix at $\theta^\star$ for the CaNN reference calibration (no regularization).

MEJORAR ESTA FIGURA

The results above define the selected CaNN benchmark used in the rest of this chapter.

6.4. PINN Pricing Performance

This section reports the baseline PINN pricing quality before introducing Sobolev terms in the training objective. The goal is to establish a reproducible reference for price-level accuracy and runtime against the COS/Heston engine.

6.4.1. Reproducible Benchmark Setup

The pricing benchmark compares a baseline parametric PINN against the COS/Heston reference under a common evaluation protocol and shared numerical conventions.

The evaluated model is the baseline parametric PINN (without Sobolev terms), benchmarked against the Heston-COS reference. Inputs follow the 8-dimensional convention

$$[\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r],$$

with affine standardization fitted on training statistics. Concretely, each feature x_j is transformed as

$$\tilde{x}_j = \frac{x_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}},$$

where μ_j^{train} and σ_j^{train} are computed only on the training split and then reused unchanged in validation/test/benchmark evaluation. This is used to keep features on comparable scales and to stabilize training and gradient computation, which is especially important in PINN settings; it also avoids data-leakage from evaluation sets. The evaluation sample contains 10,000 candidates, with 9,996 valid comparisons after numerical filtering. COS pricing is computed with $N = 1500$, $L = 50$, and fallback $L = 80$, under European put convention ($K = 1$) and no-arbitrage tolerance 10^{-8}. All reported metrics are computed in double precision on CPU.

6.4.2. Global Pricing Metrics

Table 6.8 reports the compact global summary used in this chapter.

TABLE 6.8. GLOBAL PINN PRICING METRICS VERSUS COS REFERENCE.

Metric	**Value**
n evaluated points	9,996
MSE	3.1621×10^{-5}
RMSE	5.6233×10^{-3}
MAE	2.7458×10^{-3}
p_{95} Abs. Err.	1.1840×10^{-2}
MAPE (%)	16.0084
R^2	0.99957

Source: Own elaboration

These results are obtained with the baseline PINN only (without Sobolev augmentation). In runtime terms, the PINN evaluates the full benchmark in 0.0203 s ($\approx 4.91 \times 10^5$ points/s), while COS requires 114.47 s ($\approx$ 87.36 points/s), i.e., an empirical throughput ratio of approximately 5.63×10^3.

6.4.3. Spatial And Distributional Diagnostics

The pointwise agreement between PINN and COS prices is first assessed in Figure 6.6. The cloud remains tightly concentrated around the identity line ($R^2 = 0.99957$), while a mild negative bias (mean error -1.87×10^{-3}) indicates slight underpricing in the central price range.

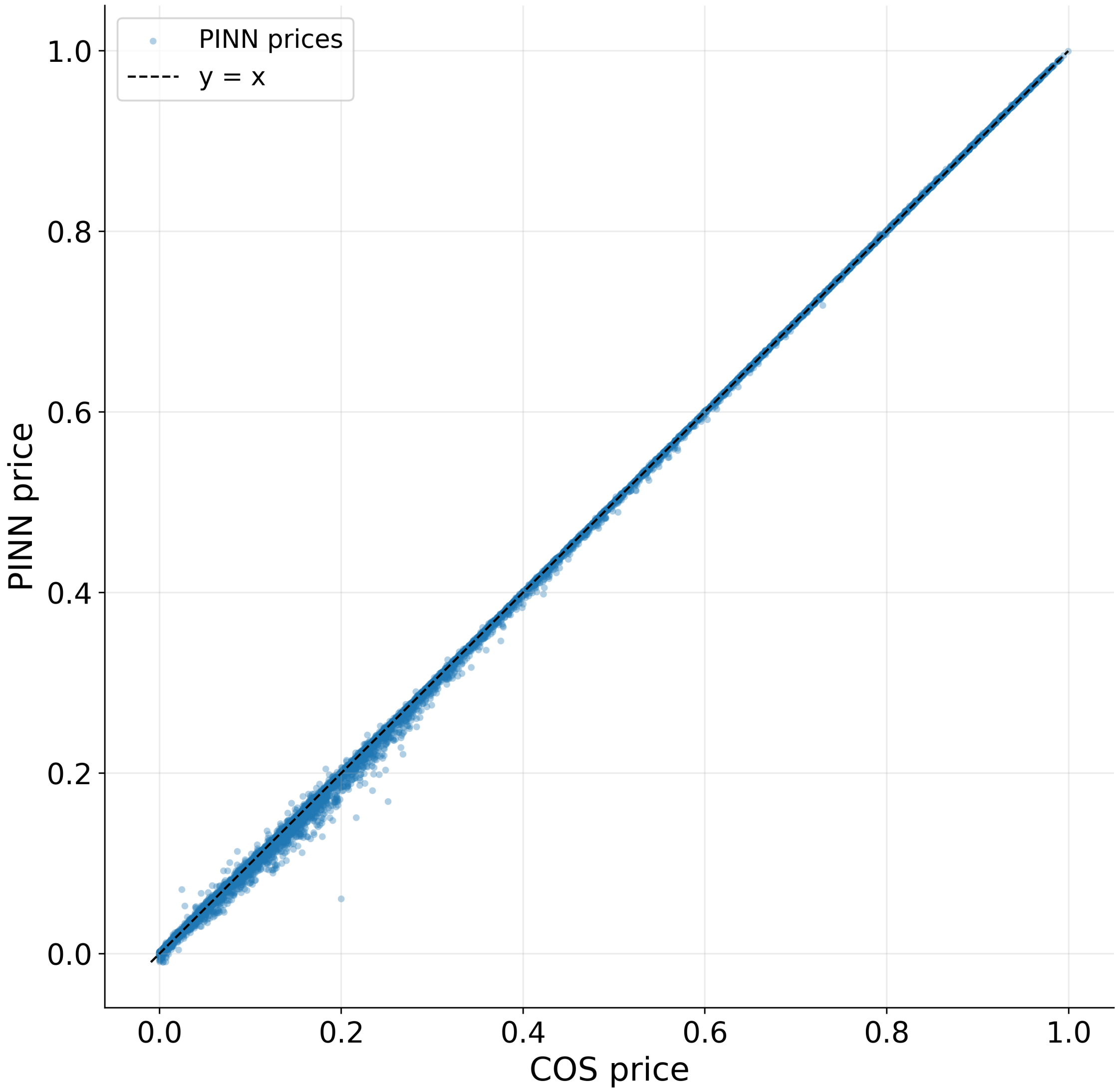

Fig. 6.6. PINN price predictions versus COS reference values on the benchmark set.

The spatial structure of absolute pricing error is shown in Figure 6.7. Error is not homogeneous over (m, τ): the largest deviations are concentrated in long maturities and moneyness tails, especially for $m > 1.2$ and $\tau > 2$, whereas the central region is markedly more stable.

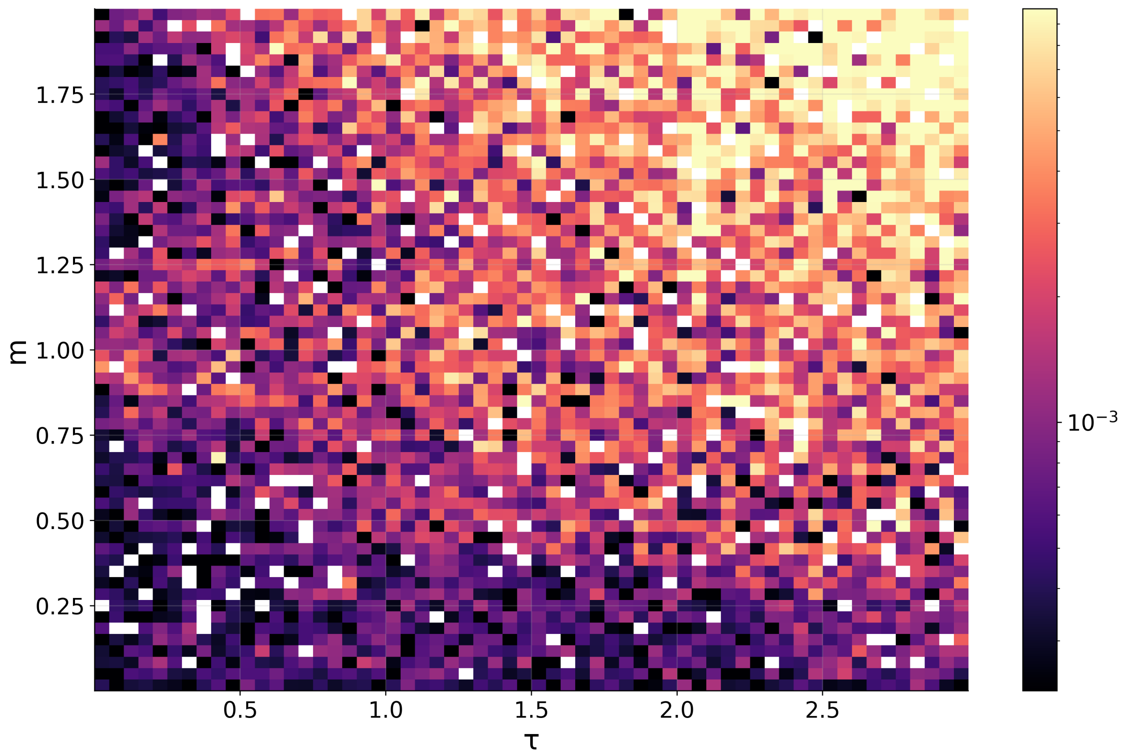

Fig. 6.7. Absolute pricing-error map over the (m, τ) benchmark grid.

SUAVIZAR?

Relative error patterns are presented in Figure 6.8. These values must be interpreted jointly with absolute errors: when the COS reference price is close to zero, relative error inflates, so local high percentages do not necessarily correspond to large monetary mispricing.

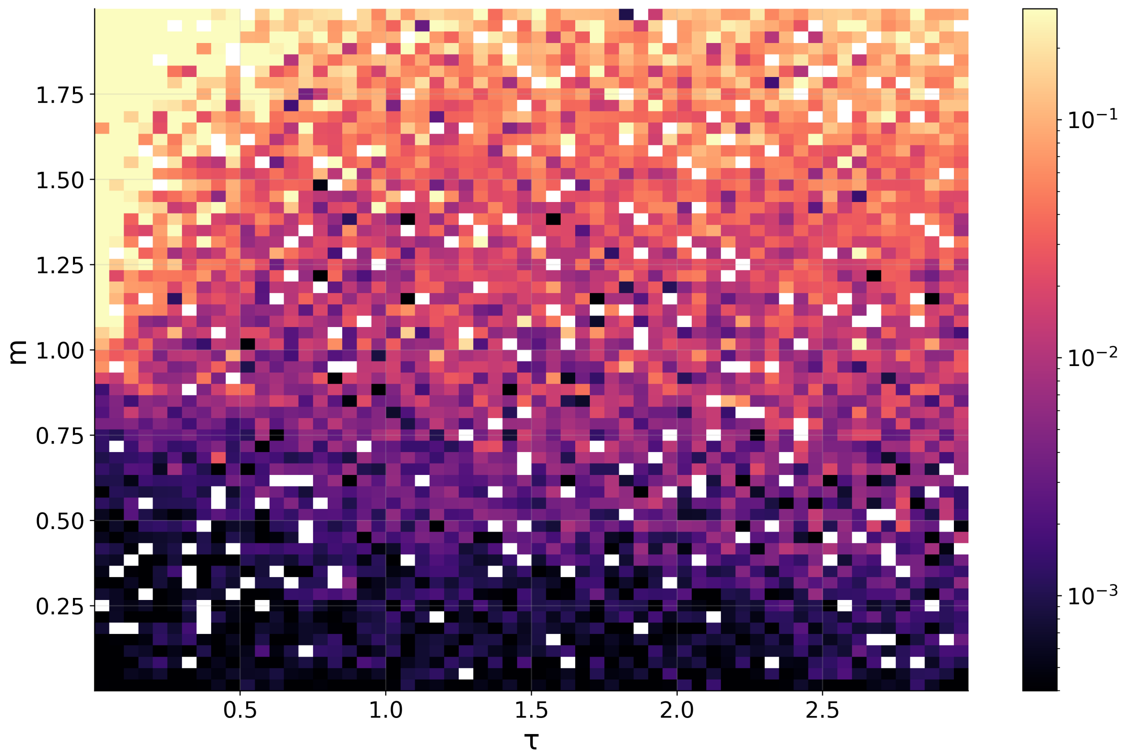

Fig. 6.8. Relative absolute pricing-error map over the (m, τ) benchmark grid.

SUAVIZAR?

The global distribution of absolute errors is summarized in Figure 6.9. The median absolute error is 1.02×10^{-3}, $p_{95} = 1.18 \times 10^{-2}$, and only about 1.7% of points exceed 2×10^{-2}, which confirms high overall accuracy with a limited but non-negligible tail.

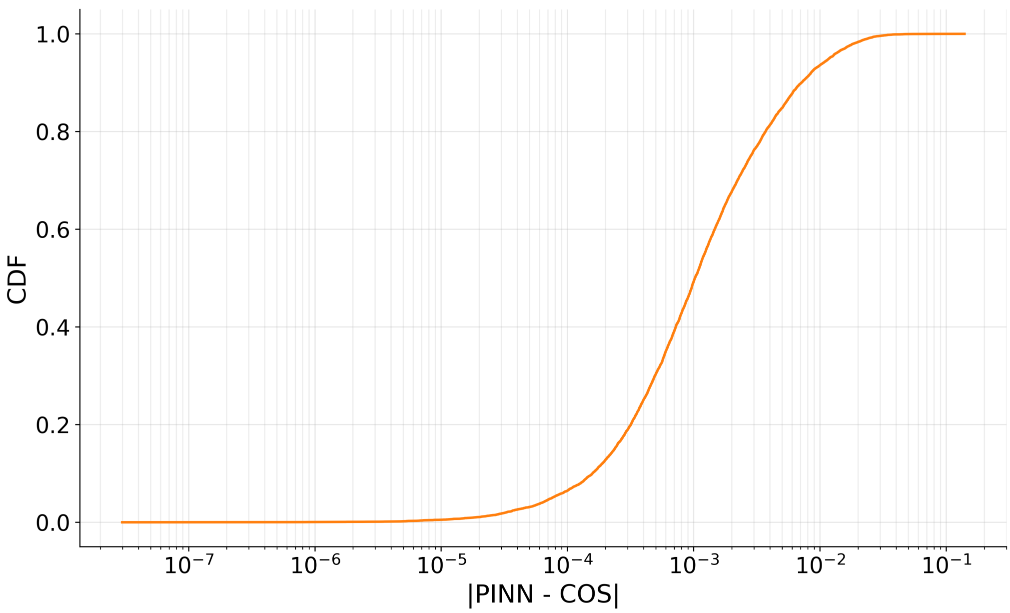

Fig. 6.9. Empirical CDF of absolute pricing errors for the baseline PINN.

6.5. PINN Greeks Performance

This section evaluates autodiff-based Greek quality for the baseline PINN and quantifies the effect of Sobolev augmentation through a controlled ablation.

6.5.1. Ablation Design And Reproducible Protocol

The ablation isolates the impact of Jacobian-informed Sobolev terms on top of the baseline PINN objective, comparing:

- **Baseline PINN**: PDE + terminal + lower-boundary losses only.
- **Sobolev-augmented PINN**: baseline loss plus derivative supervision terms $\mathcal{L}_{g,1}$ and $\mathcal{L}_{g,2}$ as defined in Chapter 5.

To keep the comparison interpretable, both variants are benchmarked on the same (m, τ) grid, with identical fixed parameters, the same Heston characteristic-function Greek engine, and the same numerical integration settings.

TABLE 6.9. REPRODUCIBLE PROTOCOL FOR THE GREEK ABLATION BENCHMARK.

Item	Value
Compared variants	Baseline PINN vs Sobolev-augmented PINN
Architecture and features	Fixed across variants (same parametric PINN input convention)
Evaluation grid	$m \in [0.8, 1.2]$ (161 points), $\tau \in [0.05, 2.0]$ (161 points)
Total evaluation size	$n = 25{,}921$ points per Greek
Fixed model parameters	$v = 0.04,\ \rho = -0.7,\ \kappa = 2.0,\ \gamma = 0.3,\ \bar{v} = 0.04,\ r = 0.01$
Reference Greeks	Semi-analytical Heston characteristic-function Greeks
Characteristic function integration setup	$u_{\min} = 10^{-6}$, $u_{\max} = 200$, $n_u = 1200$
Shared conventions	Put option, strike $K = 1$, THETA sign convention (THETA $= -\partial V/\partial\tau$), MAPE floor 10^{-4}
Arithmetic context	Double precision on CPU

Source: Own elaboration

6.5.2. Ablation Metrics By Greek

Greek quality is reported with the metrics exported by the benchmark pipeline: RMSE, R^2, MAPE (%), and MAE.

TABLE 6.10. GREEK METRICS FOR BASELINE AND SOBOLEV PINN (N = 25,921 PER GREEK).

Variant	Greek	RMSE	R^2	MAPE (%)	MAE
Baseline	DELTA	3.33×10^{-2}	0.9831	10.74	2.82×10^{-2}
Baseline	GAMMA	3.85×10^{-1}	0.8592	82.98	2.85×10^{-1}
Baseline	VEGA	6.55×10^{-2}	0.5534	71.84	5.80×10^{-2}
Baseline	THETA	4.82×10^{-3}	0.9270	31.98	3.10×10^{-3}
Baseline	RHO	8.37×10^{-2}	0.9431	95.00	7.16×10^{-2}
Sobolev	DELTA	3.51×10^{-4}	1.0000	0.28	2.41×10^{-4}
Sobolev	GAMMA	8.94×10^{-3}	0.9999	31.29	4.29×10^{-3}
Sobolev	VEGA	3.17×10^{-3}	0.9990	36.11	1.70×10^{-3}
Sobolev	THETA	5.30×10^{-4}	0.9991	1.88	1.27×10^{-4}
Sobolev	RHO	1.43×10^{-3}	1.0000	9.05	1.02×10^{-3}

Source: Own elaboration

Compared with the baseline PINN, Sobolev augmentation reduces RMSE by approximately 9× (THETA) to 95× (DELTA), with large R^2 gains for the two hardest Greeks in the baseline model (GAMMA and VEGA). Greek-inference throughput decreases from 9.48×10^3 to 8.60×10^3 points/s (~ 9.3% reduction), which remains a clearly favorable trade-off given the global accuracy gains.

6.5.3. Visual Diagnostics

This subsection is organized as a before/after narrative. Baseline diagnostics are first used to localize the dominant Greek-error modes, and Sobolev diagnostics are then read with the same visual layout to make improvements directly comparable.

Figure 6.10 displays baseline relative absolute-error maps for the five Greeks. The spatial pattern is heterogeneous, with broad high-error regions in GAMMA and VEGA, which is consistent with their weaker baseline R^2 values in Table 6.10.

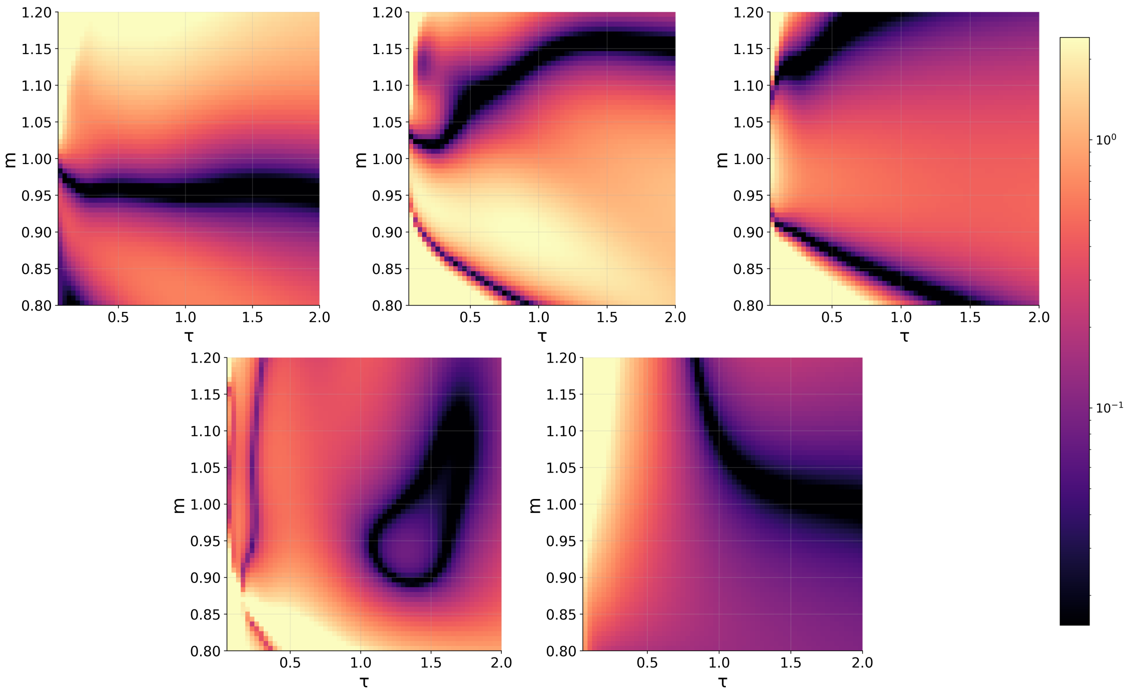

Fig. 6.10. Relative absolute-error maps for baseline PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO).

MEJORAR ESTA FIGURA

The baseline histogram panel complements this map-level view with error distributions. The heaviest tails appear in GAMMA and VEGA, while DELTA, THETA, and RHO are comparatively tighter. This confirms that the baseline model captures coarse sensitivity structure but still exhibits non-negligible extreme deviations in the hardest derivatives.

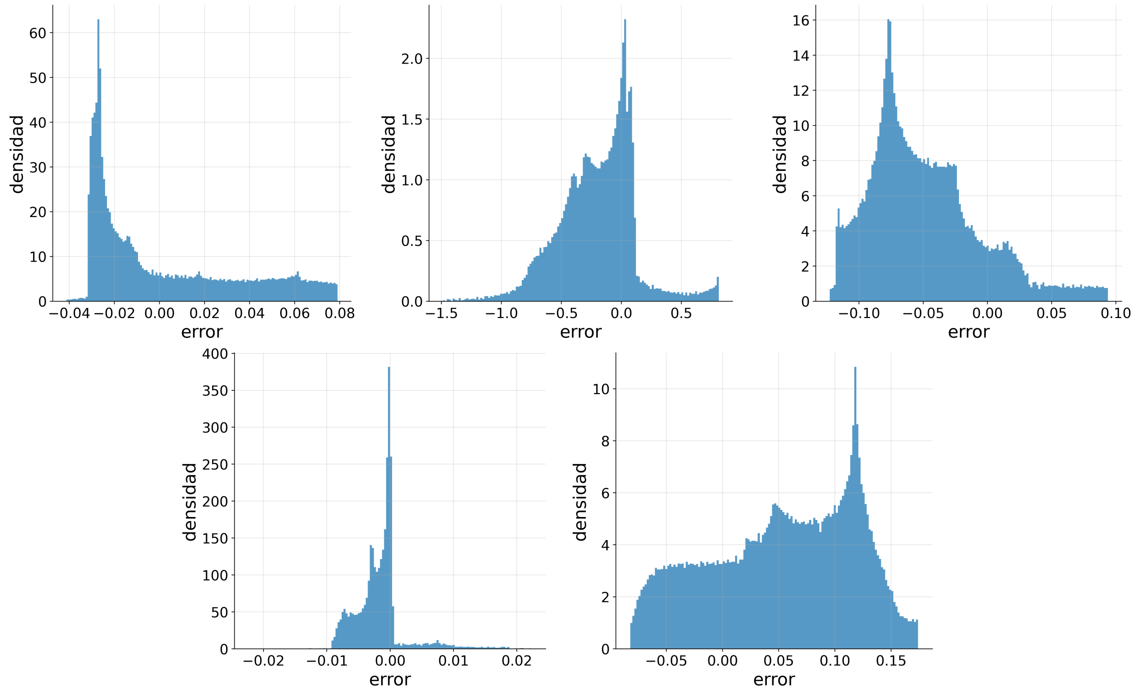

Fig. 6.11. Error histograms for baseline PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO).

MEJORAR ESTA FIGURA

After adding Sobolev terms, Figure 6.12 shows a clear contraction of high-error regions across all Greeks. The strongest visual improvement appears in GAMMA and VEGA, where problematic zones become much smaller and less intense than in the baseline maps. This spatial change is consistent with the order-of-magnitude RMSE reductions reported in Table 6.10.

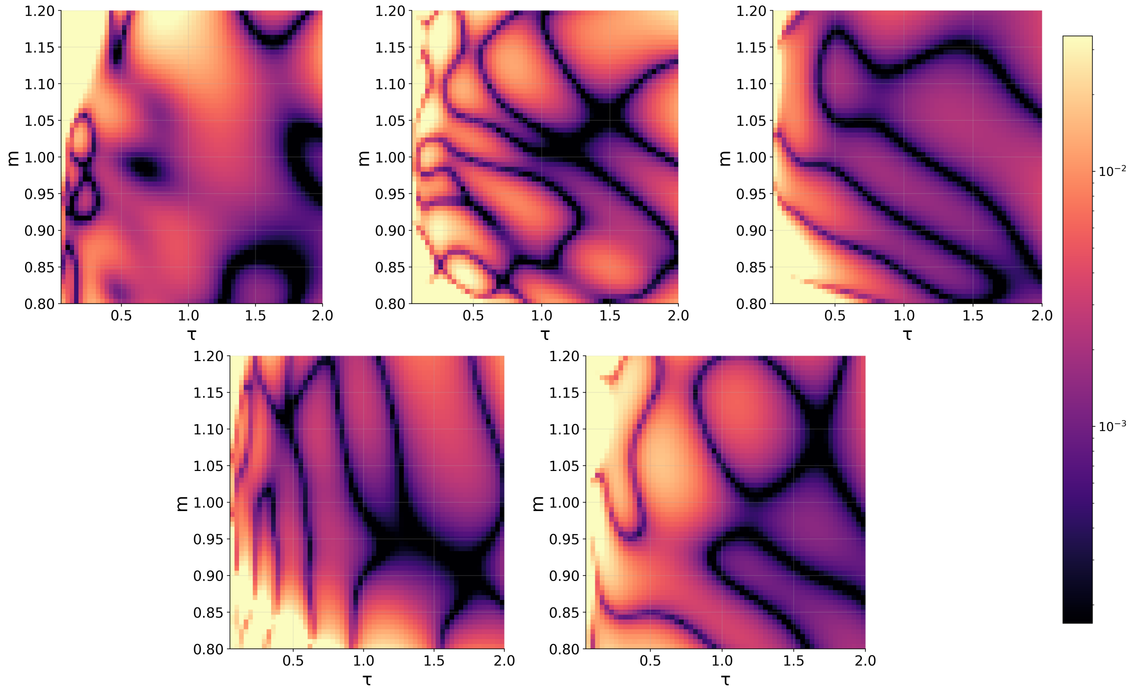

Fig. 6.12. Relative absolute-error maps for Sobolev-augmented PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO).

MEJORAR ESTA FIGURA

The same effect appears in Figure 6.13: Sobolev training narrows the distributions and reduces tail mass, so large deviations become less frequent for all five Greeks compared with the baseline histogram panel. The tail compression is particularly visible in GAMMA and VEGA, where baseline outliers were most persistent.

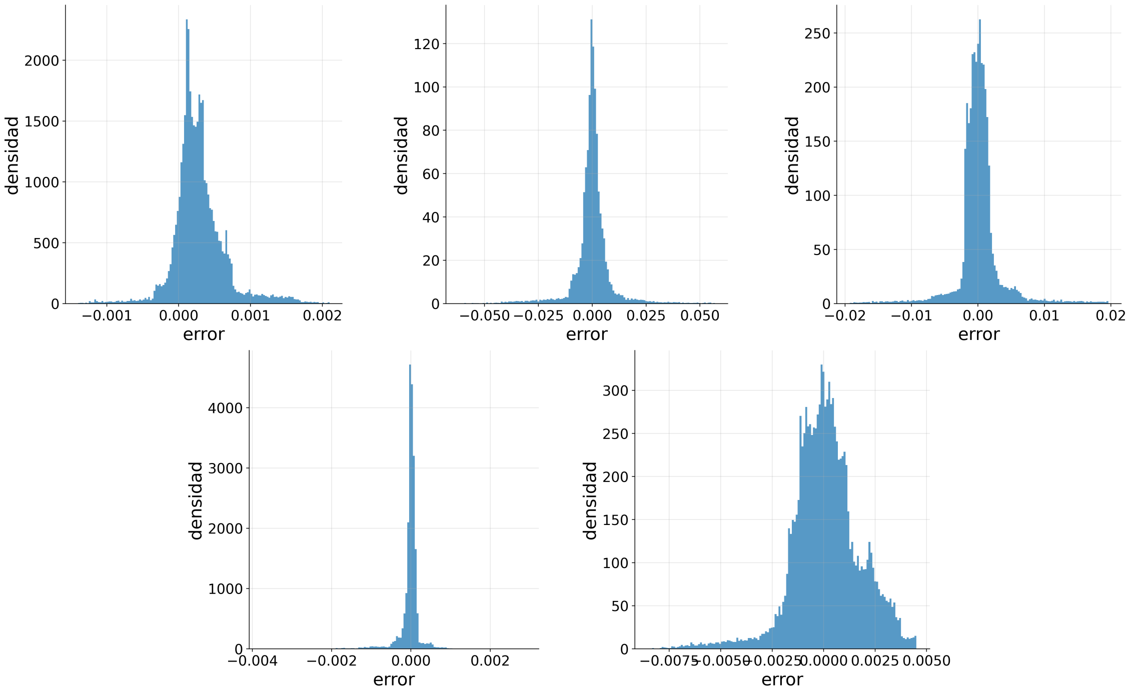

Fig. 6.13. Error histograms for Sobolev-augmented PINN Greeks (DELTA, GAMMA, VEGA, THETA, RHO).

MEJORAR ESTA FIGURA

Figure 6.14 shows the two-phase optimization schedule used in this ablation: epochs 0 to 4000 are trained without Sobolev terms, and Sobolev augmentation is activated from epoch 4000 onward. Together, the five diagnostics indicate that Sobolev augmentation improves local derivative fidelity, not just aggregate scalar scores, while keeping the runtime overhead limited.

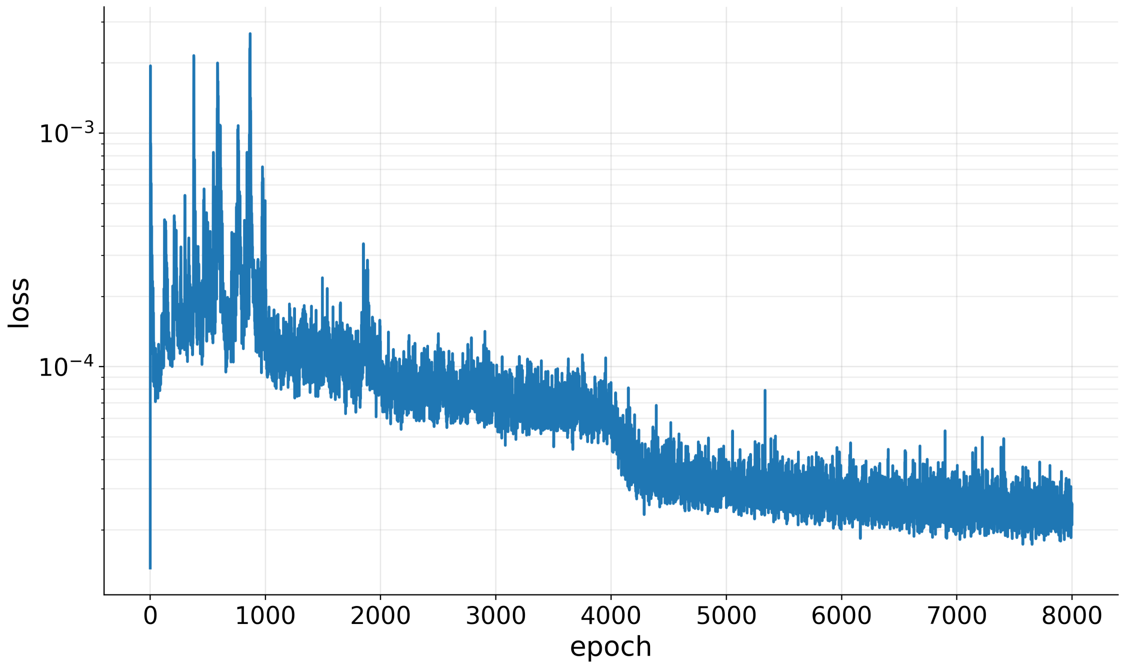

Fig. 6.14. Training loss trajectory with phase switch at epoch 4000 (without Sobolev before 4000, with Sobolev from 4000 onward).

6.6. Cross-Model Computational Comparison

This section summarizes the end-to-end pipeline in operational order, emphasizing where each model contributes and how computational cost is distributed.

First, market calibration is driven by the ANN pricer plus the CaNN optimization loop. The ANN stage provides fast implied-volatility evaluations ($\approx 8.43 \times 10^5$ points/s), while CaNN concentrates cost in iterative inverse fitting (134 iterations, 261 objective evaluations) and reaches residual RMSE 2.01441×10^{-4} on the quote set.

Second, the calibrated market state $(\rho, \kappa, \gamma, \bar{v}, v_0)$ is injected into the parametric PINN pricing stage. In this configuration, baseline PINN pricing reaches RMSE 5.62×10^{-3} with $R^2 = 0.99957$, and runs at $\approx 4.91 \times 10^5$ points/s. Under the same benchmark protocol, COS runs at ≈ 87.36 points/s, so PINN remains several orders of magnitude faster in repeated evaluation mode.

Third, Greek extraction is performed on top of the same calibrated parameter state. The baseline PINN Greek pass runs at $\approx 9.48 \times 10^3$ points/s, while the Sobolev variant runs at $\approx 8.60 \times 10^3$ points/s (about 9.3% slower) and substantially improves derivative accuracy (for example, GAMMA RMSE from 3.85×10^{-1} to 8.94×10^{-3}, VEGA RMSE from 6.55×10^{-2} to 3.17×10^{-3}).

Pipeline-level error should be read as stagewise bounds, not as a single scalar. The ANN and CaNN stages control IV-space mismatch used for calibration, pricing quality is validated in price space against COS, and Greek quality is validated in derivative space.

Since these are different output spaces and objectives, the most reliable reading is the per-stage error envelope reported above and in Sections 6.2–6.5.

7. CONCLUSION AND FUTURE WORK

7.1. Market Data

7.2. Exotic Derivatives

7.3. Greeks And Hedging

As a direct continuation of the Greek-focused PINN work, a relevant future direction is to test residual-gradient regularization in the spirit of gPINNs, i.e., adding a penalty on spatial gradients of the PDE residual. This term was not included in the benchmarked implementation of the present thesis and is therefore left for future experiments under controlled ablation settings.

7.4. Model Extension

7.5. Machine Learning Improvements

BIBLIOGRAPHY

[1] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973. DOI: 10.1086/260062.

[2] S. L. Heston, "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options," en, *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993. DOI: 10.1093/rfs/6.2.327.

[3] P. Carr and D. B. Madan, "Option valuation using the fast fourier transform," *Journal of Computational Finance*, vol. 2, no. 4, pp. 61–73, 1999. DOI: 10.21314/JCF.1999.043.

[4] F. Fang and C. W. Oosterlee, "A novel pricing method for european options based on fourier-cosine series expansions," *SIAM Journal on Scientific Computing*, vol. 31, no. 2, pp. 826–848, 2009. DOI: 10.1137/080718061.

[5] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. DOI: 10.1186/s13362-019-0066-7. Accessed: Oct. 21, 2025. [Online]. Available: https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7.

[6] D. Hainaut and A. Casas, "Option pricing in the heston model with physics inspired neural networks," en, *Annals of Finance*, vol. 20, no. 3, pp. 353–376, 2024. DOI: 10.1007/s10436-024-00452-7. [Online]. Available: https://doi.org/10.1007/s10436-024-00452-7.

[7] J. Yu, L. Lu, X. Meng, and G. E. Karniadakis, "Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114 823, 2022. DOI: 10.1016/j.cma.2022.114823. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0045782522001438.

[8] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, A3055–A3081, 2021.

[9] S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," *Journal of Computational Physics*, vol. 449, p. 110 768, 2022. DOI: 10.1016/j.jcp.2021.110768. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S002199912100663X.

[10] J. F. Urbán, P. Stefanou, and J. A. Pons, "Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?" en, *Journal of Computational Physics*, vol. 523, p. 113 656, Feb. 2025. DOI: 10.1016/j.jcp.2024.113656. Accessed: Mar. 19, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0021999124009045.

[11] B. Oksendal, *Stochastic Differential Equations: An Introduction with Applications*, 6th ed. Springer, 2003.

[12] F. D. Rouah and S. L. Heston, *The Heston Model and its Extensions in Matlab and C#*. John Wiley & Sons, 2013.

[13] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

[14] M. D. Mckay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code," en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. DOI: 10.1080/00401706.2000.10485979. Accessed: Mar. 21, 2026. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979.

[15] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989. DOI: 10.1007/BF02551274.

[16] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. DOI: 10.1016/0893-6080(91)90009-T.

[17] R. Storn and K. Price, "Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997. DOI: 10.1023/A:1008202821328.

[18] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, "Sobolev Training for Neural Networks," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper/7015-sobolev-training-for-neural-networks.

A. ANALYSIS OF THE COS METHOD ERROR

no se si dejarlo como apéndice o meterlo en alguna seccion

For the COS method, we have three sources of error:

- The integration range truncation error, ε_1.
- The series truncation error on $[a, b]$, ε_2.
- The error related to approximating $\bar{A}_k(x)$ by $\bar{F}_k(x)$, ε_3 (poner ref a alguna ecuacion).

It is important to note that there is no error in the payoff coefficient H_k for plain vanilla European options.

If the integration range $[a, b]$ is chosen sufficiently large, the overall error is dominated by ε_2 (TODO: argumentarlo). Also, ε_2 converges exponentially if $f_X(x) \in C^\infty[a, b]$ (TODO: justificarlo tamb). Otherwise, converges only algebraically.

To choose an optimal integration range we will use the expression (TODO: ref a Oorsterlee)

$$[a, b] = \left[(x + \zeta_1) - L\sqrt{\zeta_2 + \sqrt{\zeta_4}},\ (x + \zeta_1) + L\sqrt{\zeta_2 + \sqrt{\zeta_4}} \right], \tag{A.1}$$

where $L \in [6, 12]$ is a parameter that depends on the tolerance we want to have, and ζ_i are the cumulants, computed using the expression (2.8).

B. CALIBRATION OF THE HESTON MODEL

TODO:

C. GLOBAL NOTATION GLOSSARY

This appendix contains a global glossary of symbols used throughout the thesis. It is intended as a living reference and can be expanded as additional notation appears in later revisions.

TABLE C.1. GLOBAL NOTATION GLOSSARY: CORE PRICING AND MODEL NOTATION

Symbol	Meaning
$t,\ T,\ \tau$	Valuation time, maturity, and time to maturity ($\tau = T - t$).
$S_t,\ K,\ m$	Spot price, strike, and moneyness ($m = S_0/K$).
r	Constant risk-free interest rate.
$\mathbb{Q}$	Risk-neutral probability measure.
V_t	Arbitrage-free option value at time t.
$V(t_0, x)$	Option value at valuation time t_0 when the state variable is x.
$V_{BS},\ V_{\text{target}}$	Black–Scholes option value and target option value used in IV inversion.
Ψ	Option payoff function at maturity.
$X_t,\ x,\ y$	State variable and its initial/terminal values, typically $x = X(t_0)$ and $y = X(T)$.
$F_X,\ f_X$	Cumulative distribution function and probability density function of X.
θ	Generic model-parameter vector.
μ	Generic drift under the original probability measure.
$W_t^{(S)},\ W_t^{(v)}$	Brownian motions driving the spot and variance processes.
$\kappa,\ \bar{v},\ \gamma,\ \rho,\ v_0$	Heston parameters: mean reversion, long-run variance, vol-of-vol, correlation, and initial variance. Later chapters also use $\bar{v}$ and v_0 for the variance parameters.

Source: Author's elaboration

TABLE C.2. GLOBAL NOTATION GLOSSARY: TRANSFORM, COS, AND IV NOTATION

Symbol	**Meaning**
$\phi_X(u)$	Characteristic function of random variable X.
$\mathcal{M}_X(u),\ \zeta_X(u),\ \zeta_k$	Moment generating function, cumulant characteristic function, and k-th cumulant.
$u,\ u_k,\ i$	Fourier variable, COS frequency $u_k = k\pi/(b-a)$, and imaginary unit.
$[a,b],\ L,\ N,\ k$	COS truncation interval, truncation scaling parameter, number of terms, and series index.
$\bar{F}_k,\ H_k,\ U_k$	COS density, payoff, and Heston-COS payoff coefficients.
$\hat{V}_T(u),\ V_N^{\mathrm{COS}}$	Fourier transform of the maturity payoff and N-term COS price approximation.
$\varphi_H,\ C,\ D,\ d,\ g$	Heston characteristic function and its affine auxiliary functions.
$\chi_k,\ \psi_k$	Closed-form payoff integrals used in COS coefficients.
σ_{imp}	Implied volatility obtained from numerical inversion.
$F(\sigma),\ \varepsilon_{\mathrm{price}},\ \varepsilon_\sigma,\ n_{\max}$	IV root function, price tolerance, volatility tolerance, and maximum number of solver iterations.
$\hat{\sigma}_i,\ \sigma_i^{\mathrm{ref}}$	Predicted and reference implied volatilities for evaluation point i.
$\mathrm{MAE}_{IV},\ \mathrm{RMSE}_{IV}$	Mean absolute error and root mean squared error computed in implied-volatility units.
$\Pi_j,\ \Psi_j$	Heston call-pricing probability terms and their Fourier integrands.
$\Delta,\ \Gamma,\ \mathrm{Vega},\ \mathrm{Rho},\ \Theta$	Standard option Greeks: spot first derivative, spot second derivative, variance/volatility sensitivity, interest-rate sensitivity, and time sensitivity.

Source: Author's elaboration